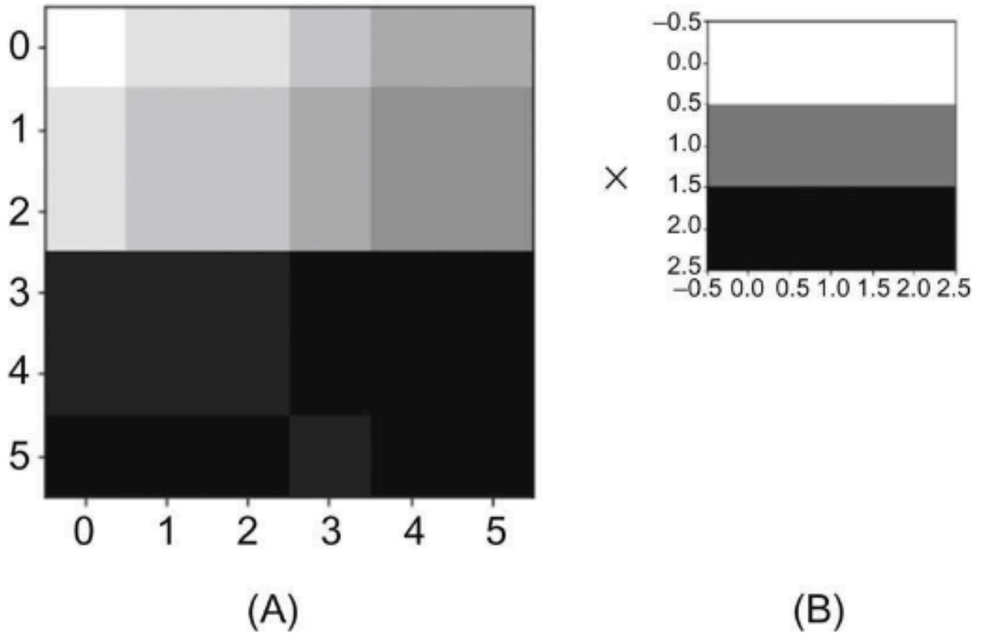
Natural Language Processing
Lab 8: Neural Models - CNN, RNN

Lonny Chen: l.chen@hertie-school.org
GRAD-E1282, Fall 2026
28-29 October 2026

Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

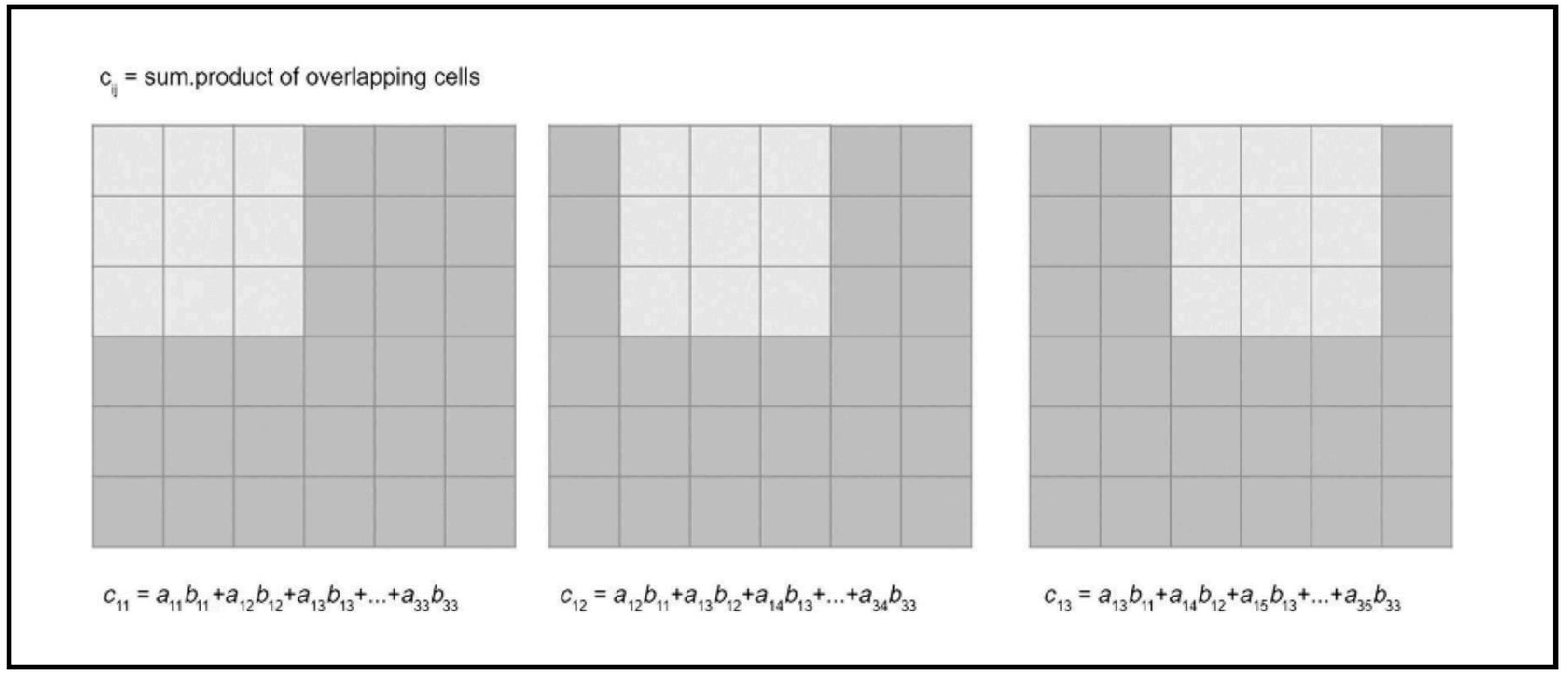


Figures: Kotu & Deshpande, 2019

Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

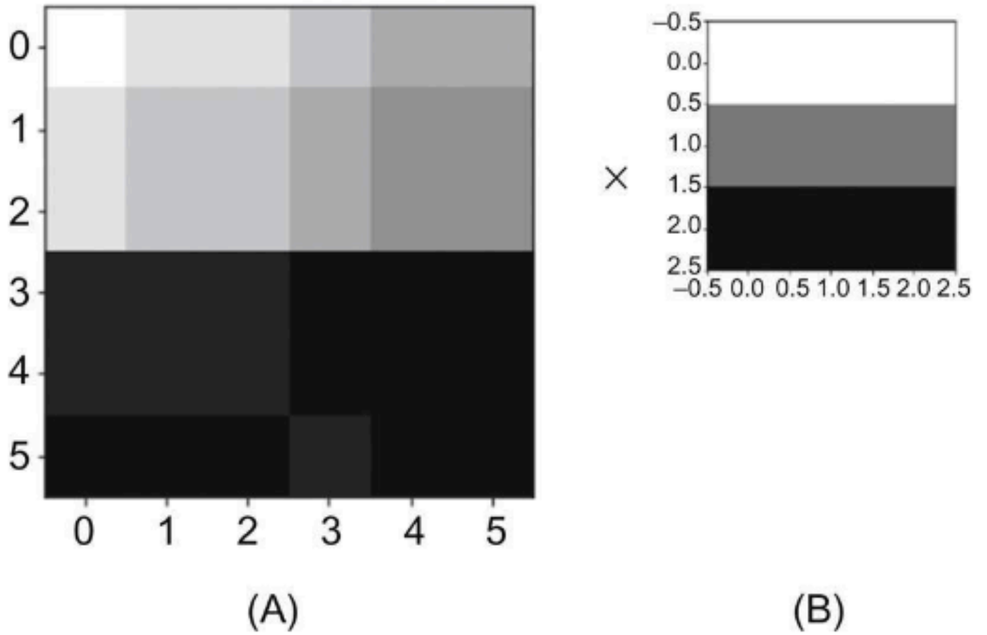


Figures: Kotu & Deshpande, 2019

Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



C

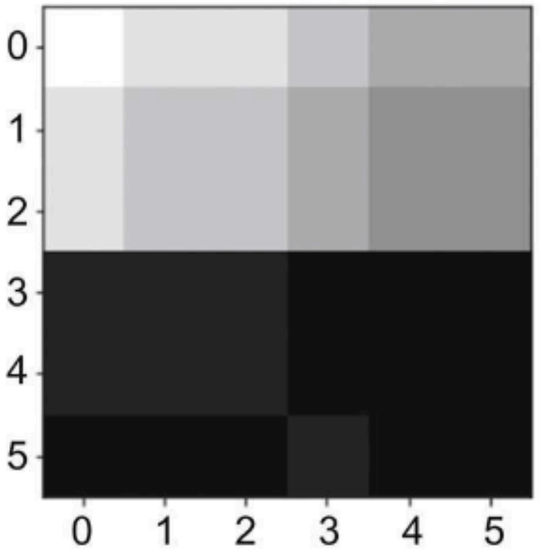
3	3	3	3
22	21	20	19
22	21	20	19
3	1	0	-1

Figures: Kotu & Deshpande, 2019

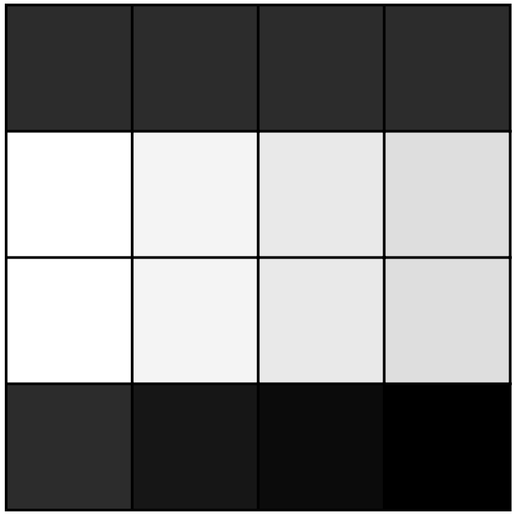
Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



C



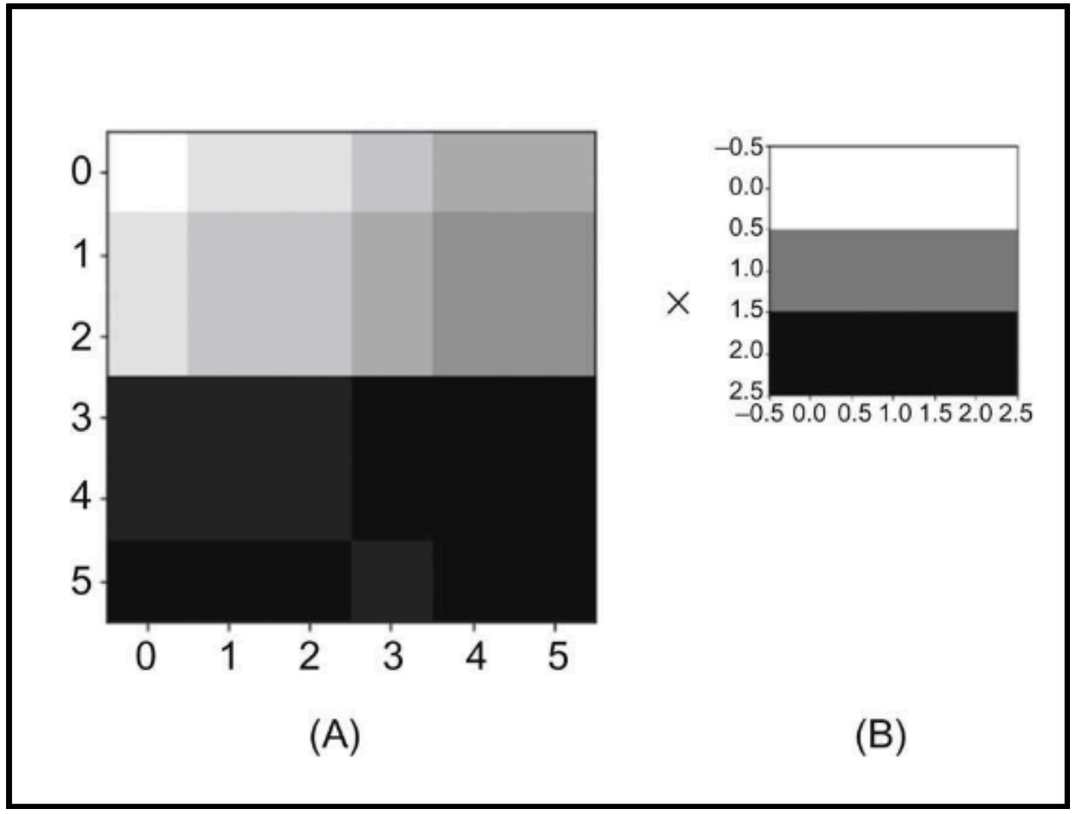
Figures: Kotu & Deshpande, 2019

Image Convolution (2D)

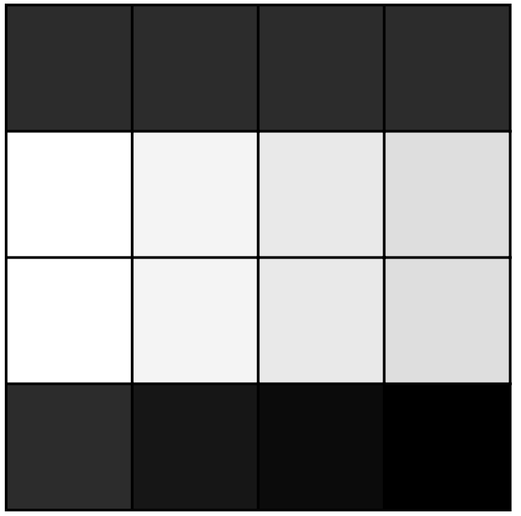
$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

n_in: 6 x 6



C

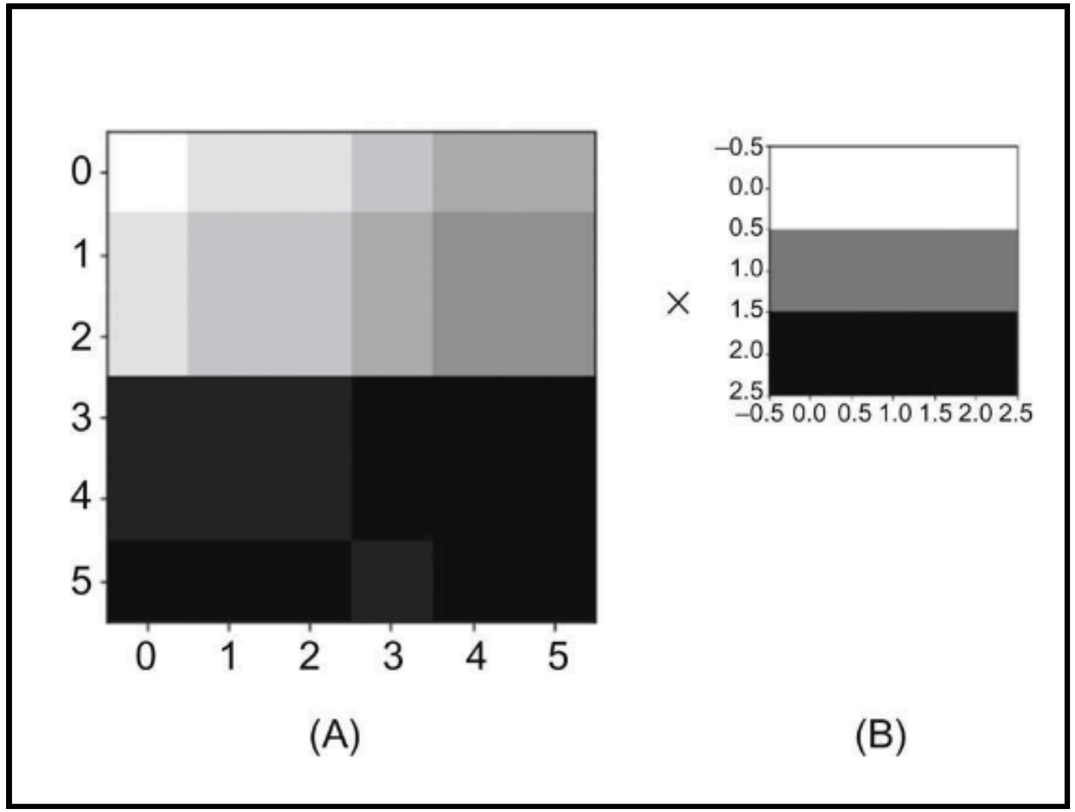


Figures: Kotu & Deshpande, 2019

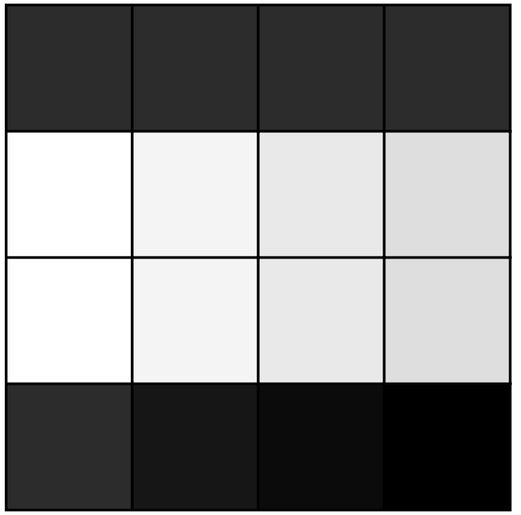
Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



C



n_in: 6 x 6

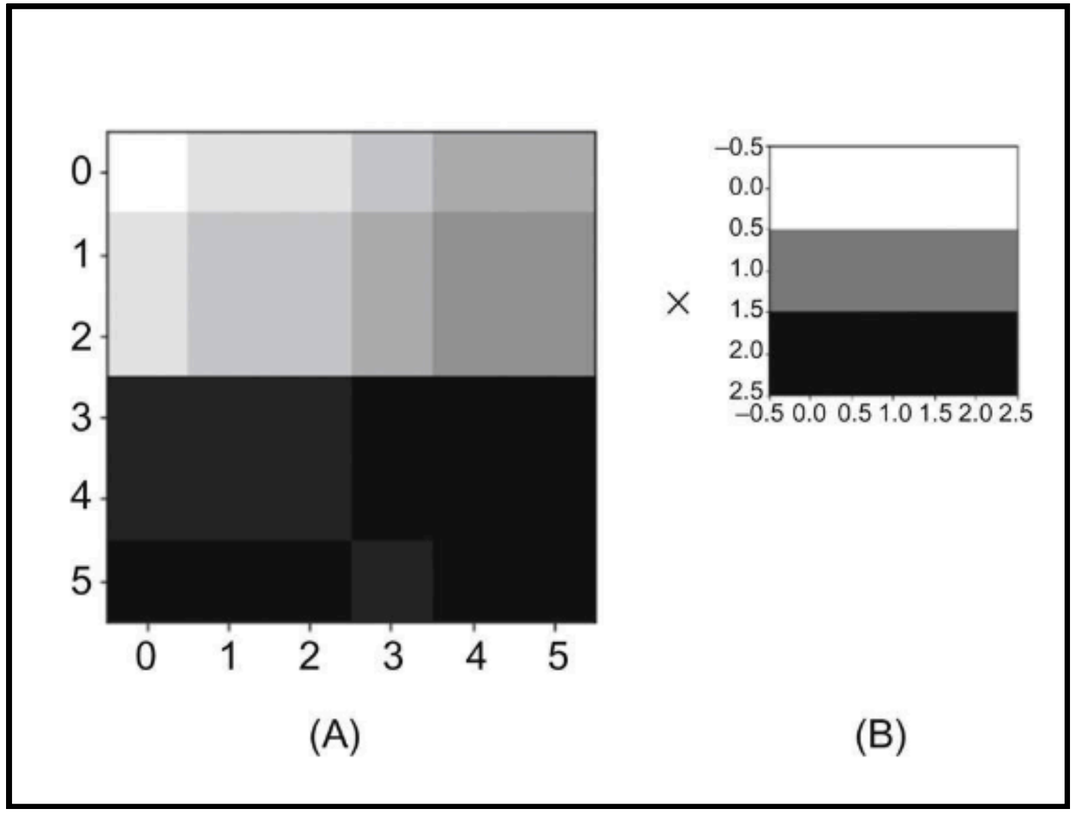
filter: 3 x 3

Figures: Kotu & Deshpande, 2019

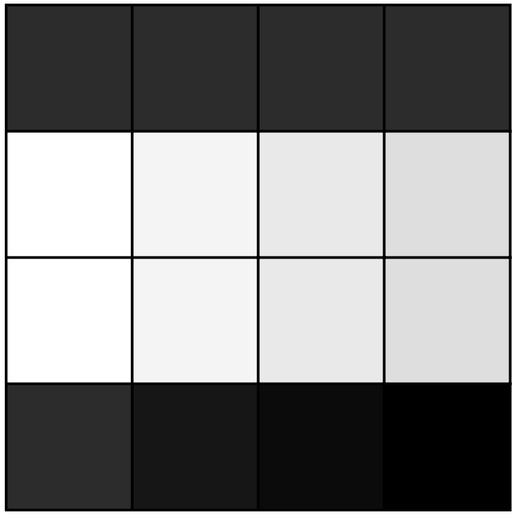
Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



C



$$n_{out} = n_{in} - f + 1$$

n_in: 6 x 6

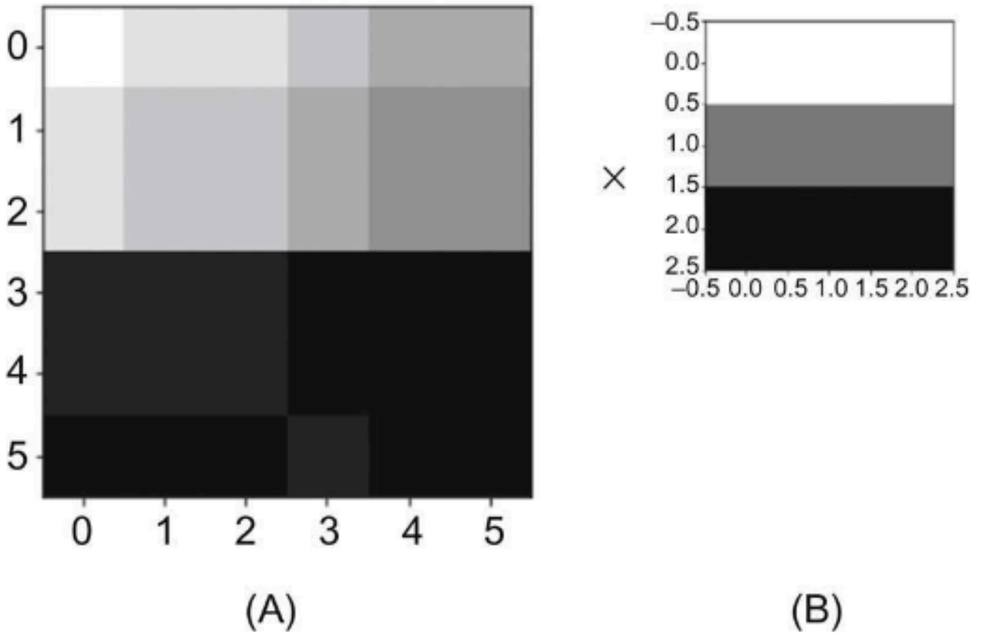
filter: 3 x 3

Figures: Kotu & Deshpande, 2019

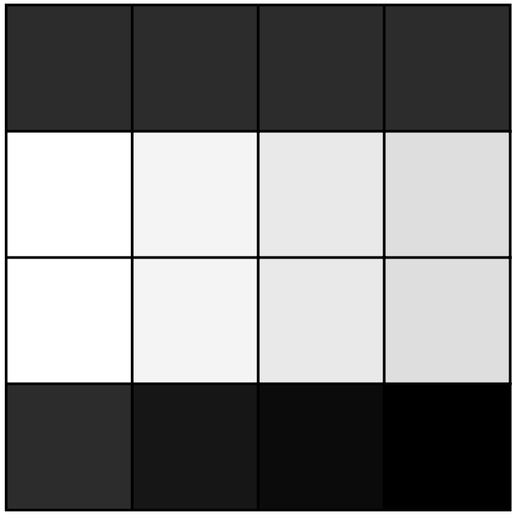
Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



C



$$n_{out} = n_{in} - f + 1$$

m in Raschka textbook

n_in: 6 x 6

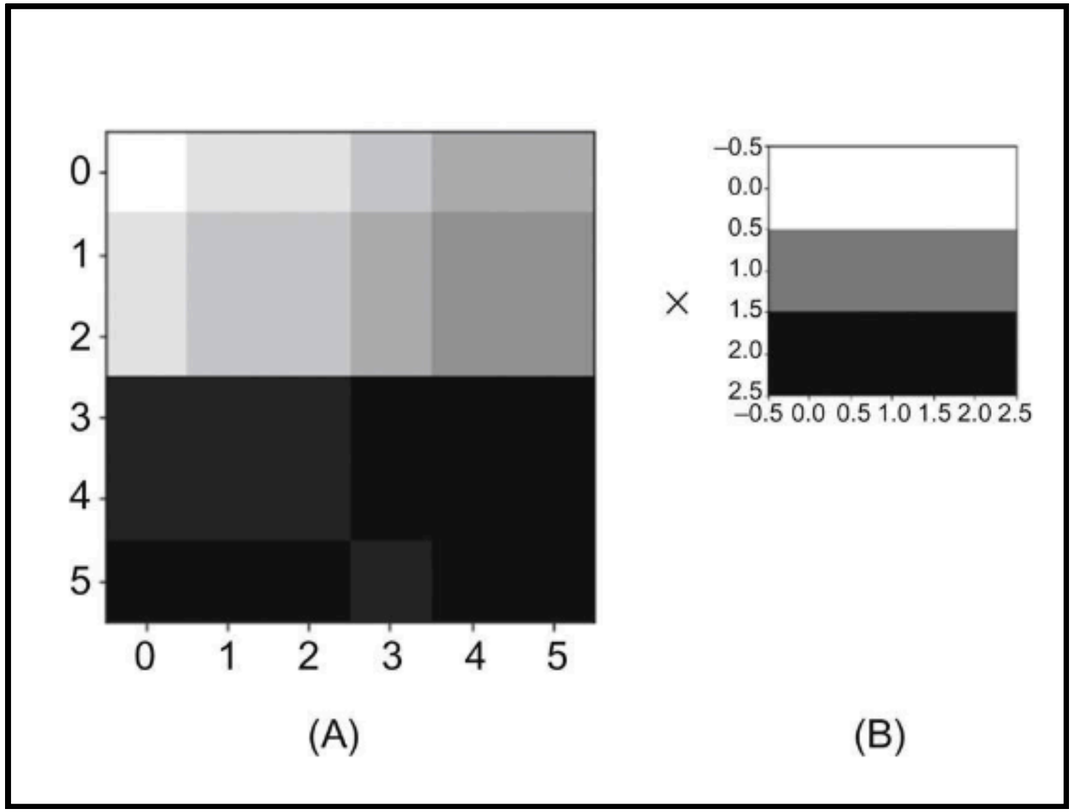
filter: 3 x 3

Figures: Kotu & Deshpande, 2019

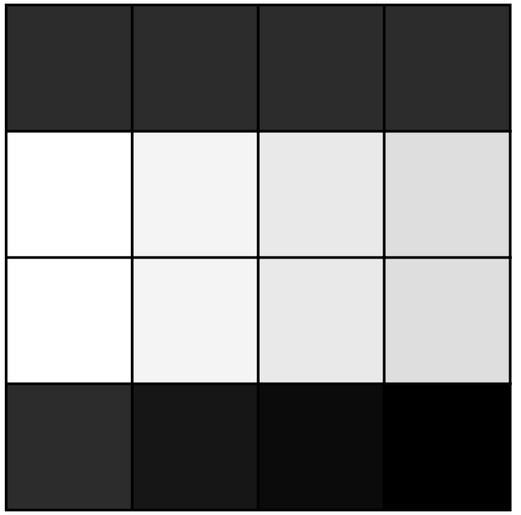
Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



C



n_in: 6 x 6

filter: 3 x 3

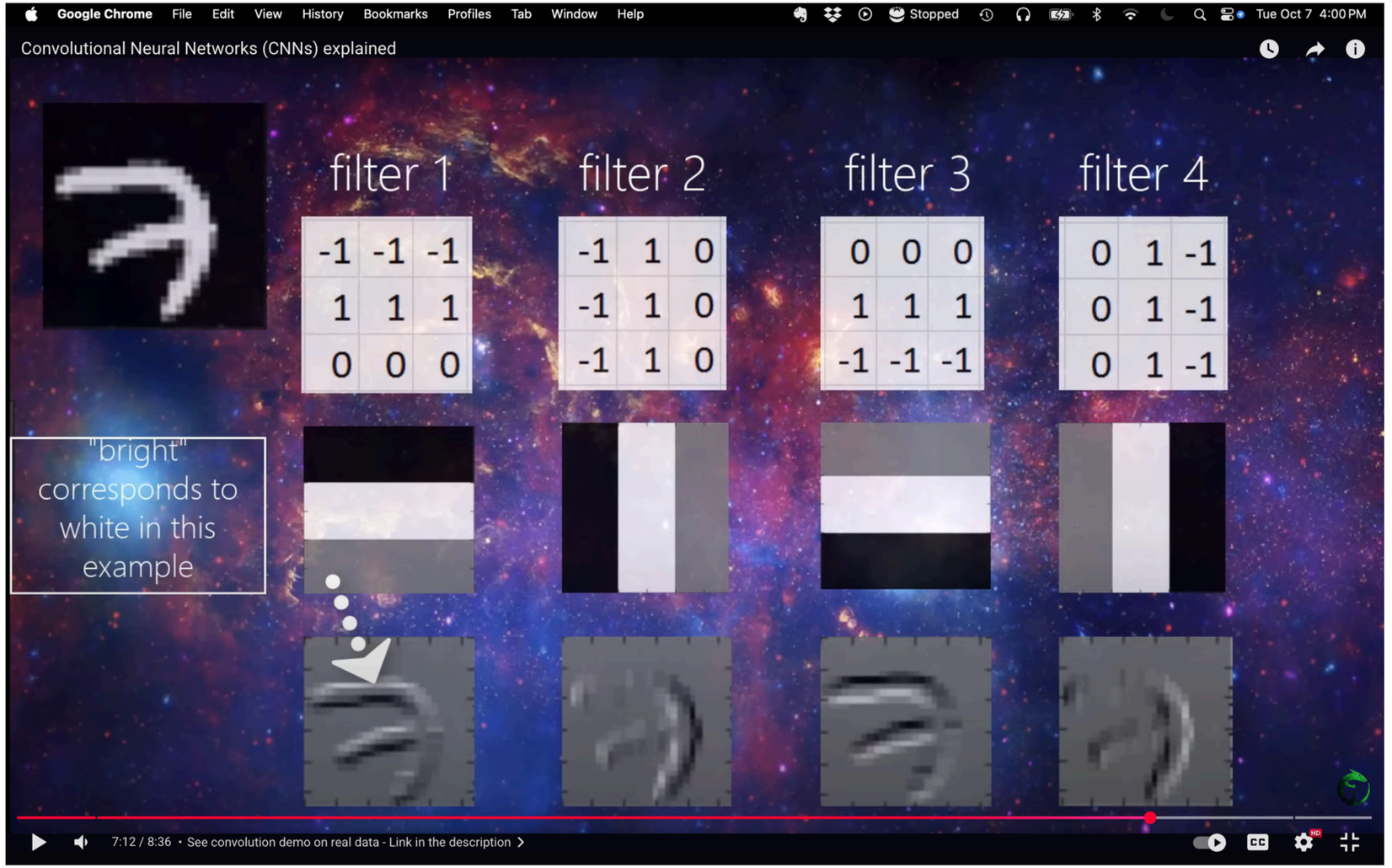
$$n_{out} = n_{in} - f + 1$$

$$= 6 - 3 + 1$$

$$= 4 \rightarrow 4 \text{ x } 4$$

Figures: Kotu & Deshpande, 2019

Image Convolution (2D)



Video frame: deeplizard, 2017

Image Convolution (2D)

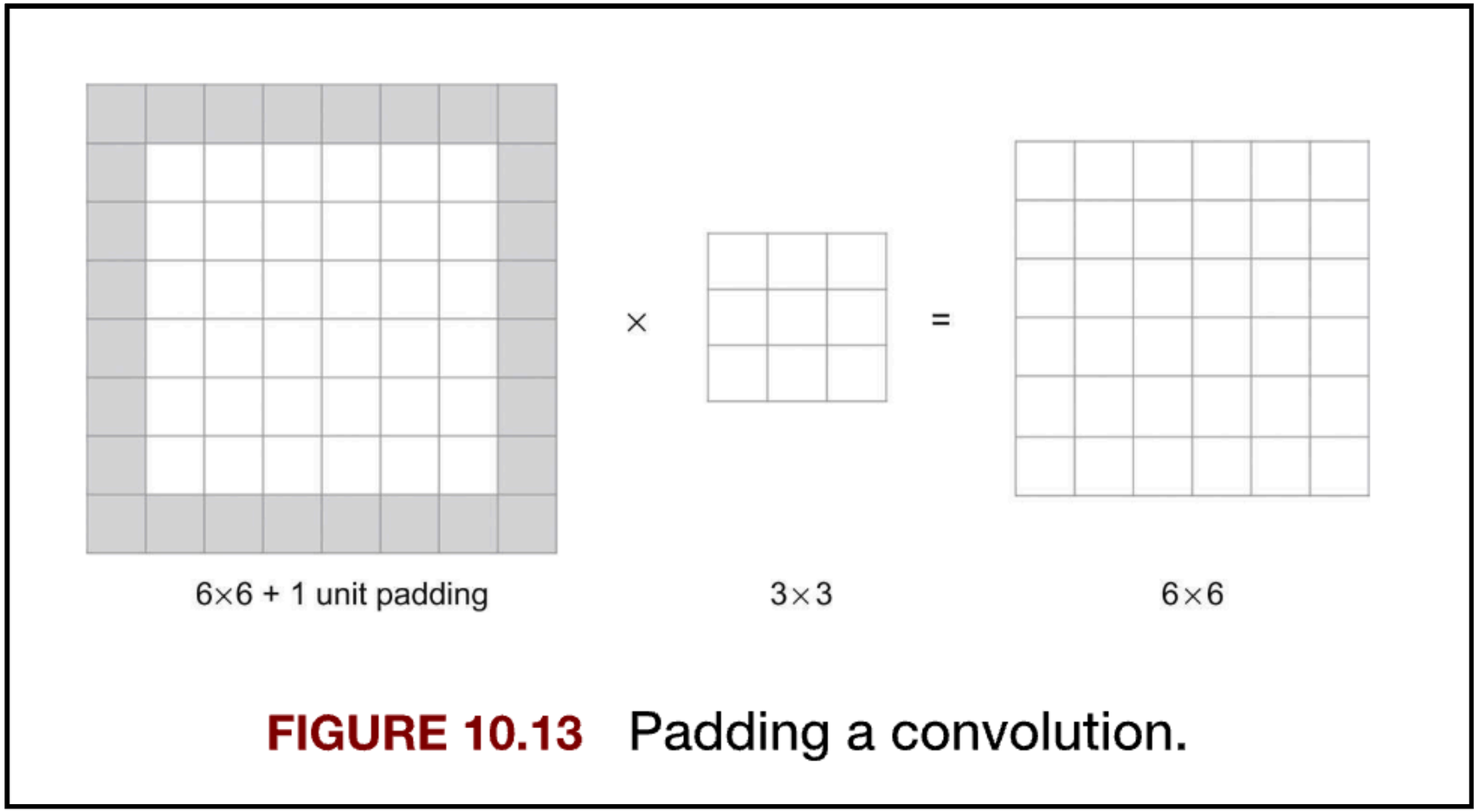


FIGURE 10.13 Padding a convolution.

Figure: Kotu & Deshpande, 2019

Image Convolution (2D)

- capture patterns at borders

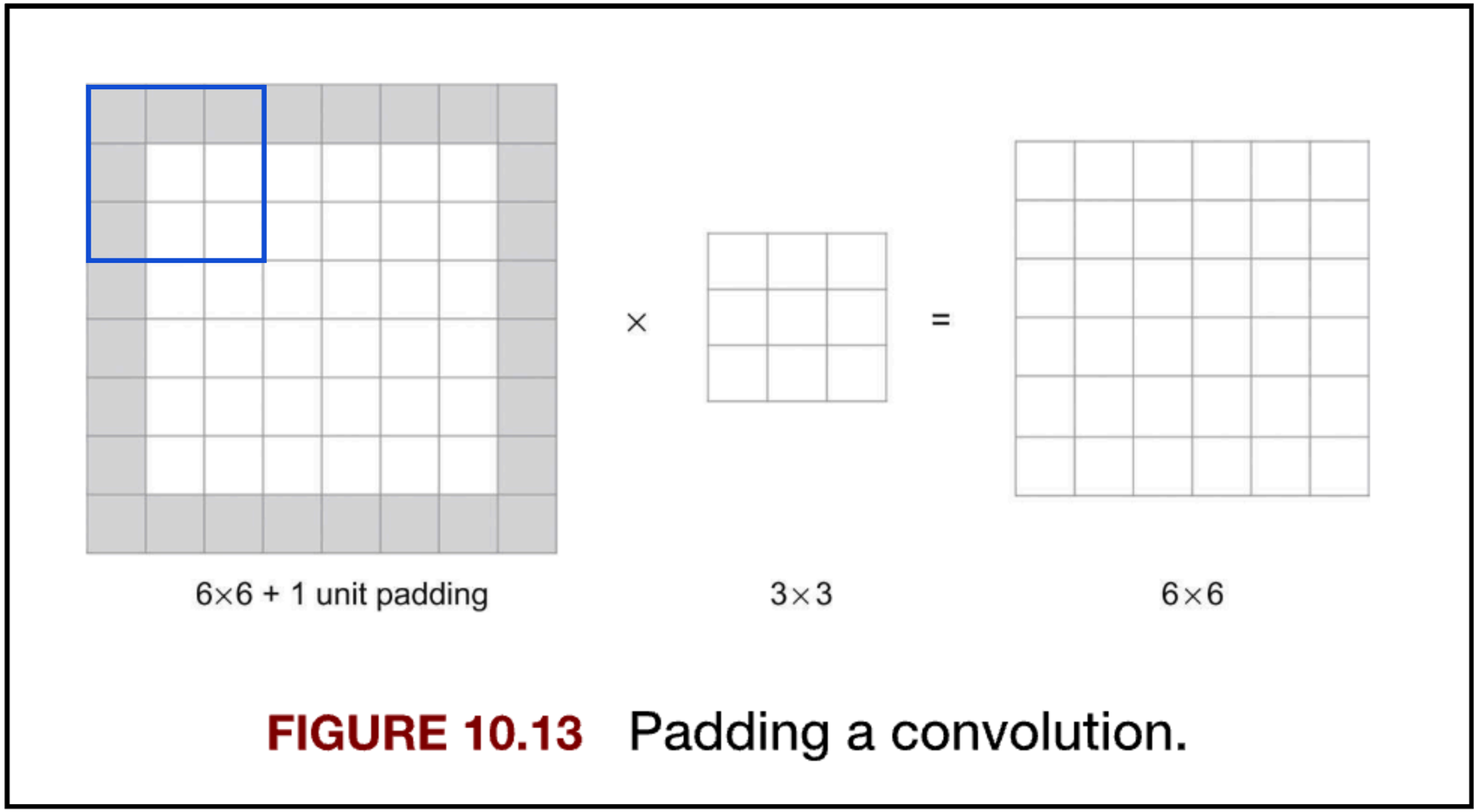


FIGURE 10.13 Padding a convolution.

Figure: Kotu & Deshpande, 2019

Image Convolution (2D)

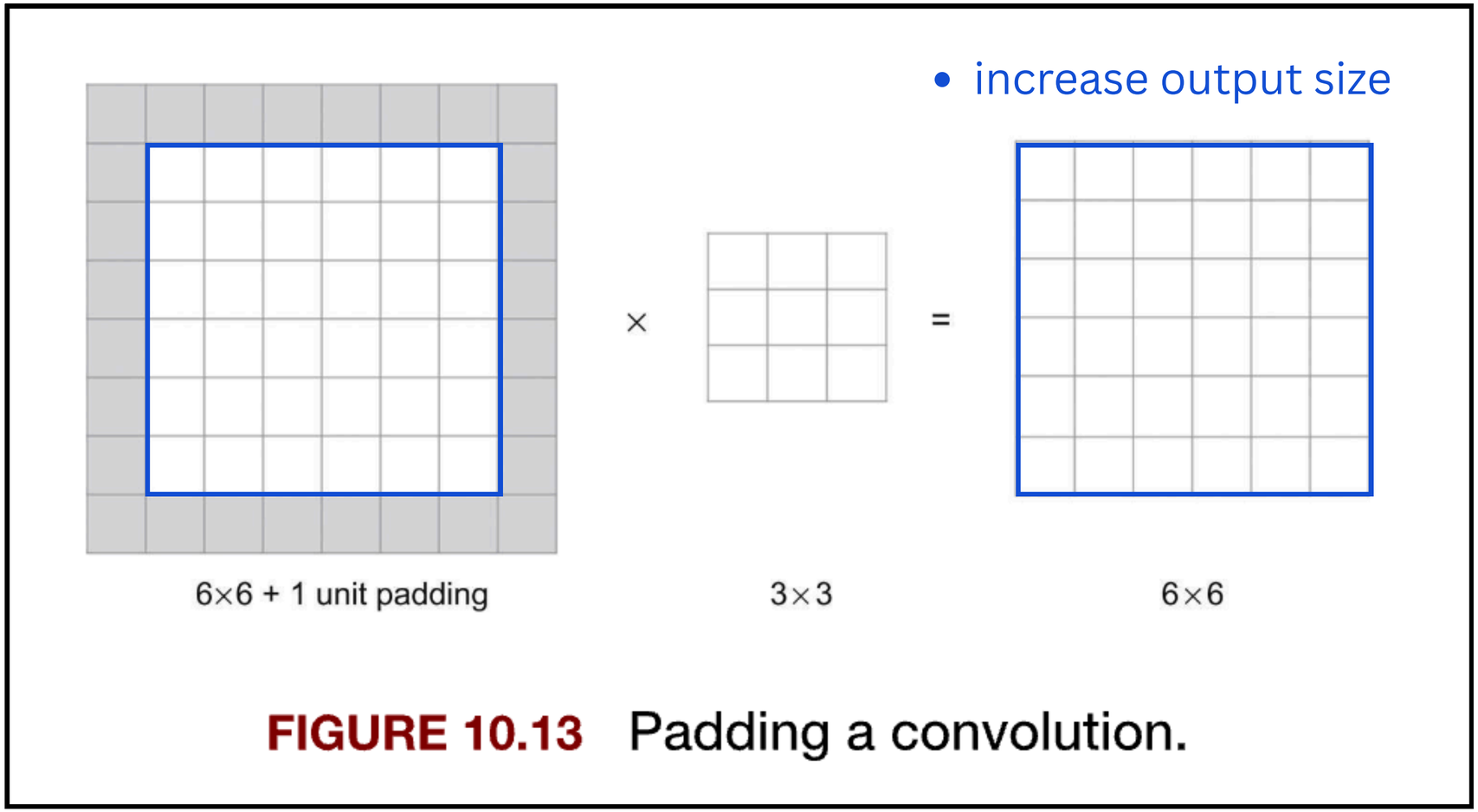
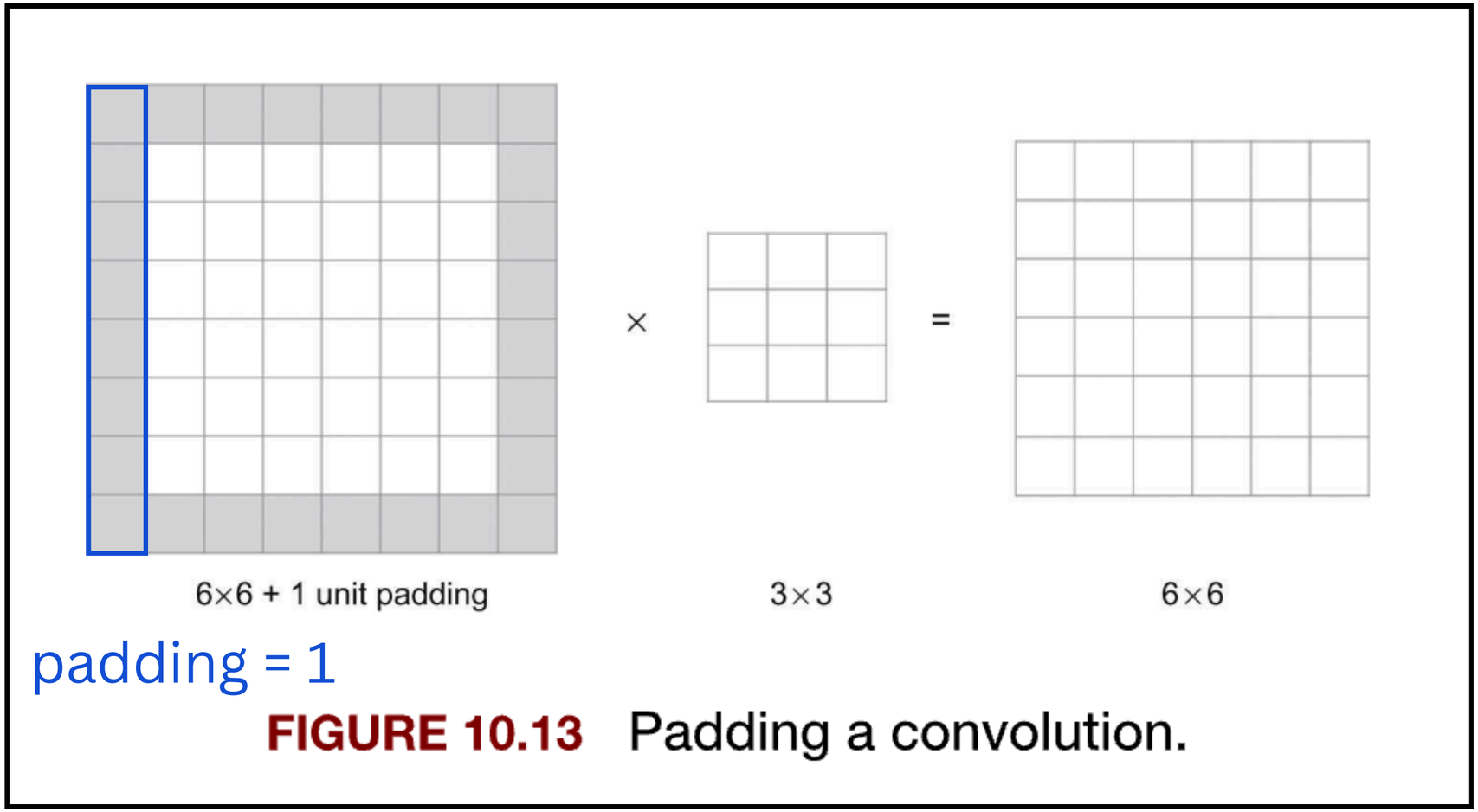


FIGURE 10.13 Padding a convolution.

Figure: Kotu & Deshpande, 2019

Image Convolution (2D)



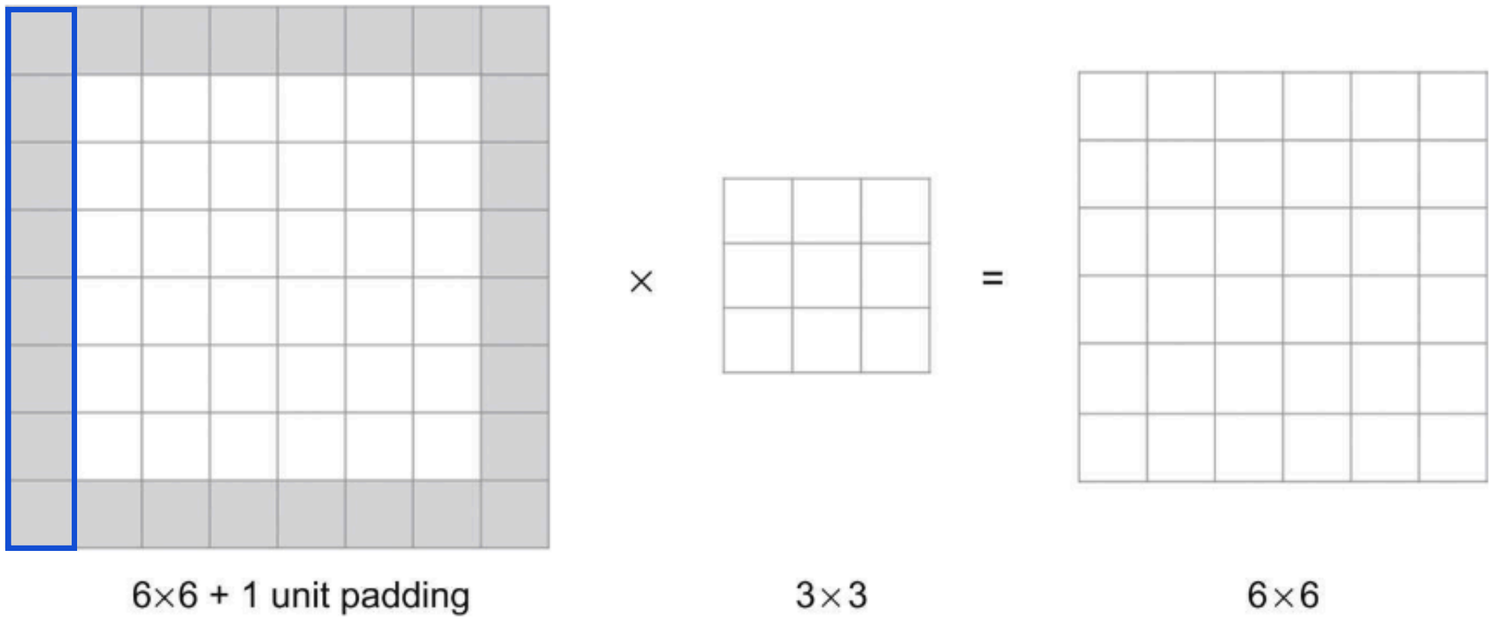
padding = 1

FIGURE 10.13 Padding a convolution.

$$n_{out} = n_{in} + 2p - f + 1$$

Figure: Kotu & Deshpande, 2019

Image Convolution (2D)



padding = 1

FIGURE 10.13 Padding a convolution.

“same” padding

$$n_{out} = n_{in} + 2p - f + 1$$
$$= 6 + 2 - 3 + 1 = 6$$

Figure: Kotu & Deshpande, 2019

Image Convolution (2D)

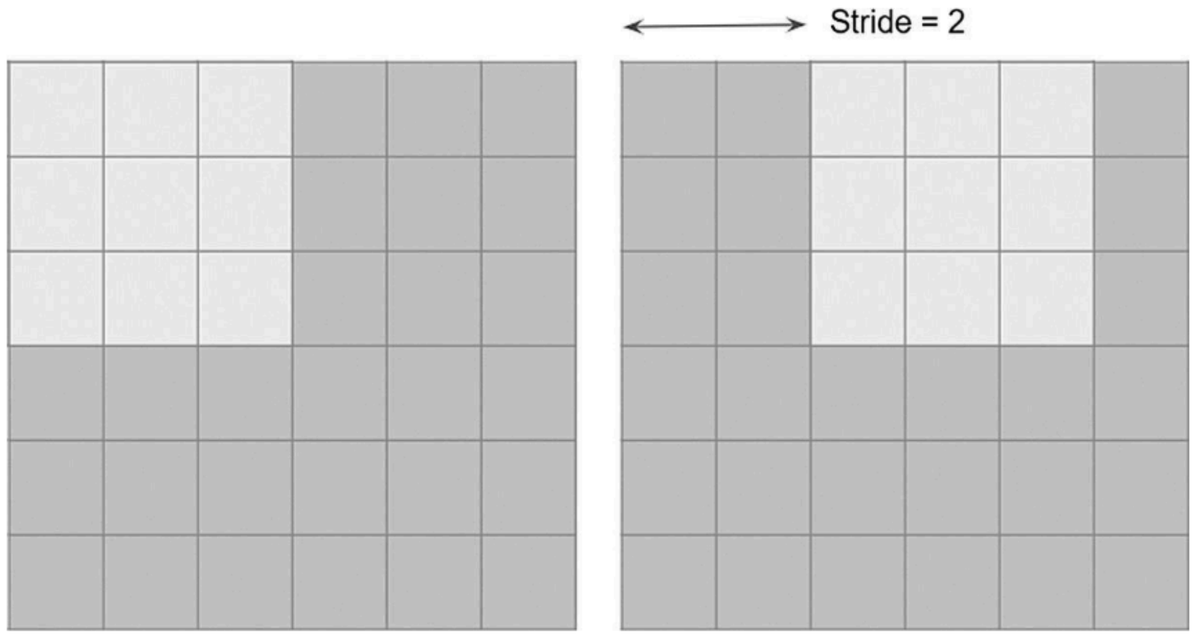


FIGURE 10.14 Visualizing the stride during a convolution.

Figure: Kotu & Deshpande, 2019

Image Convolution (2D)

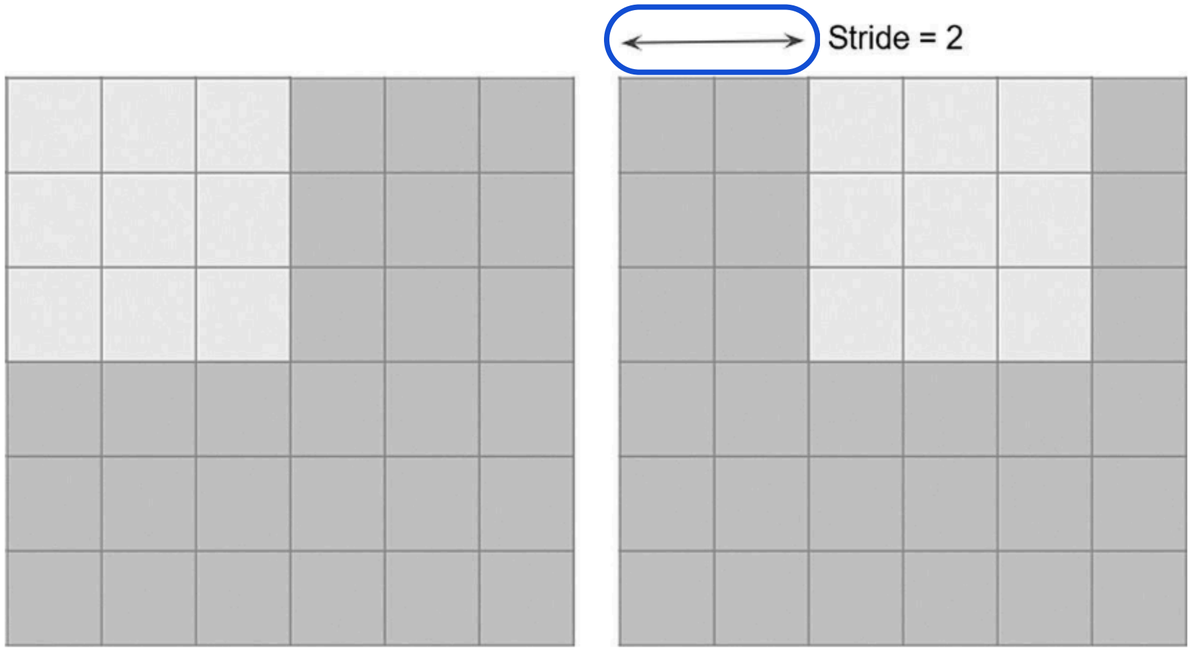


FIGURE 10.14 Visualizing the stride during a convolution.

$$\left\lfloor \frac{n_{in} + 2p - f}{s} \right\rfloor + 1 = \left\lfloor \frac{6 + 0 - 3}{2} \right\rfloor + 1 = 2$$

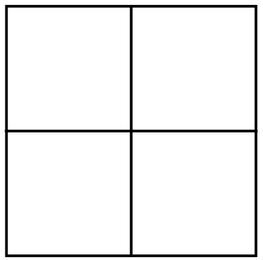


Figure: Kotu & Deshpande, 2019

Image Convolution (2D)

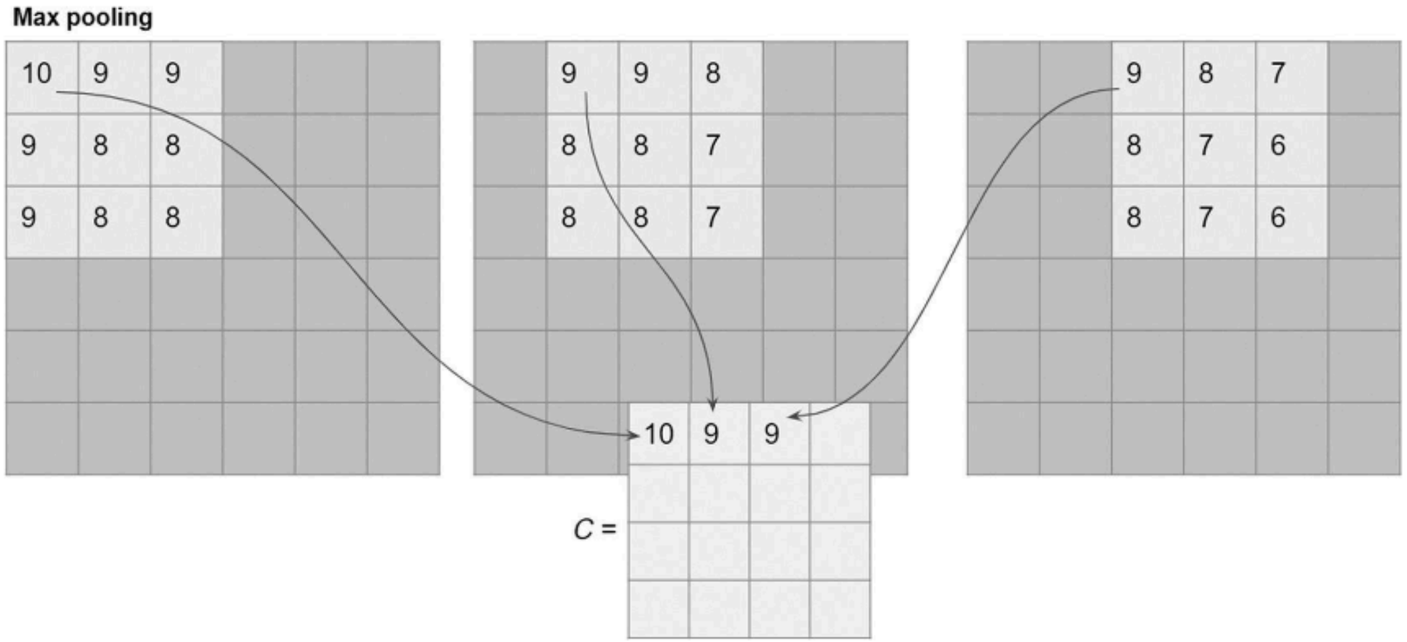


FIGURE 10.15 Max pooling visualized.

$$\left\lfloor \frac{n_{in} + 2p - f}{s} \right\rfloor + 1$$

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

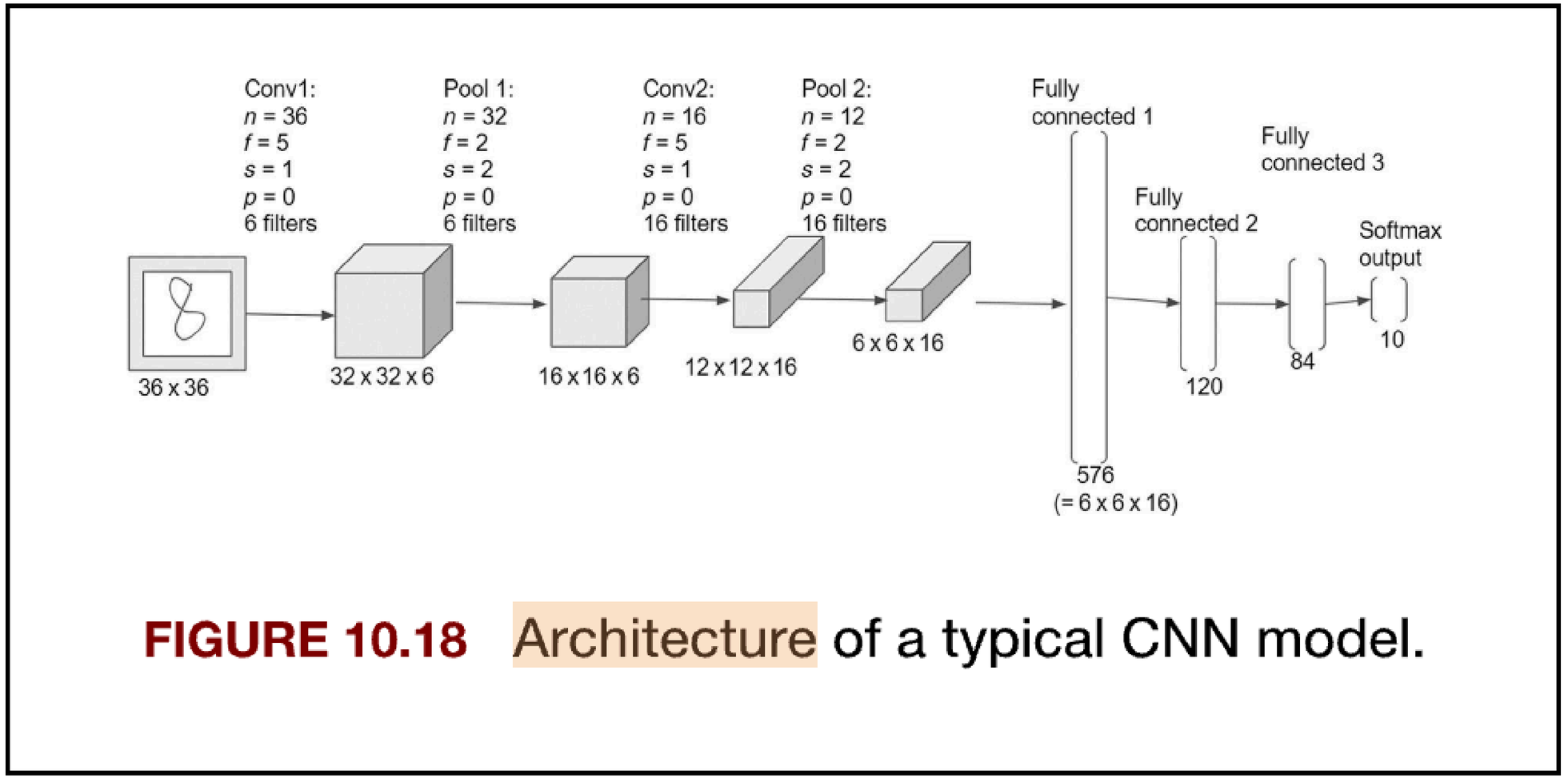


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

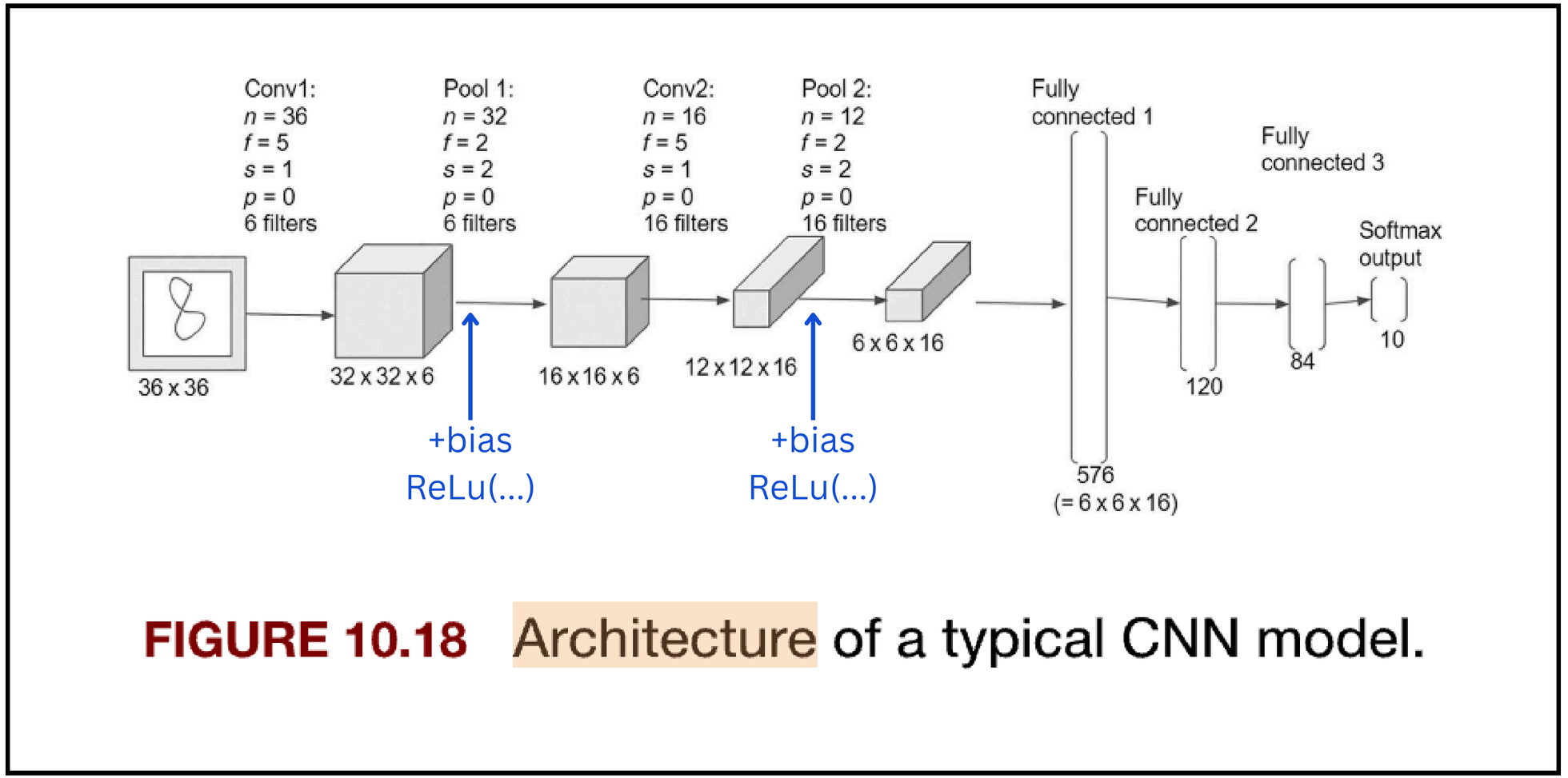


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

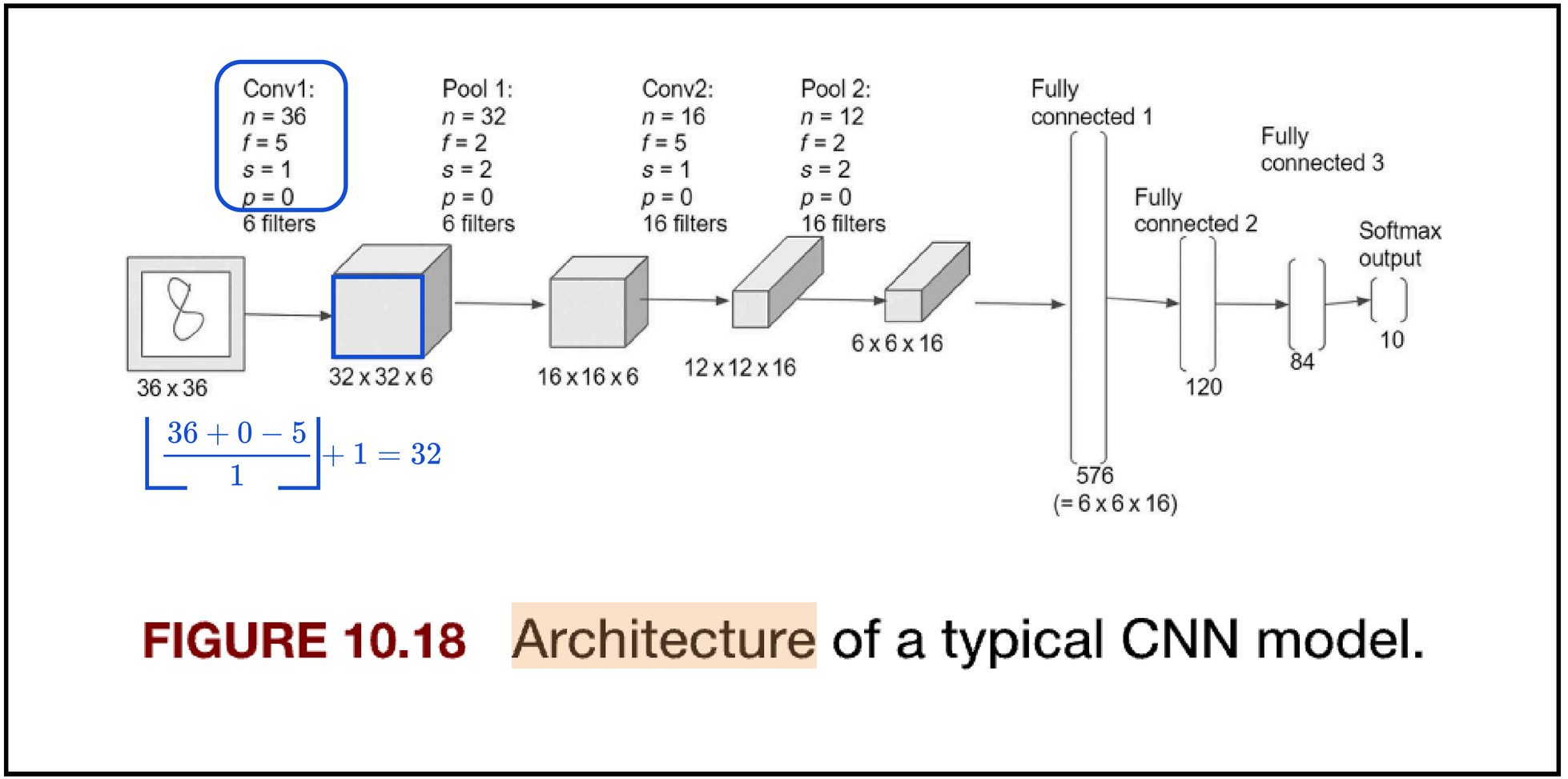


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

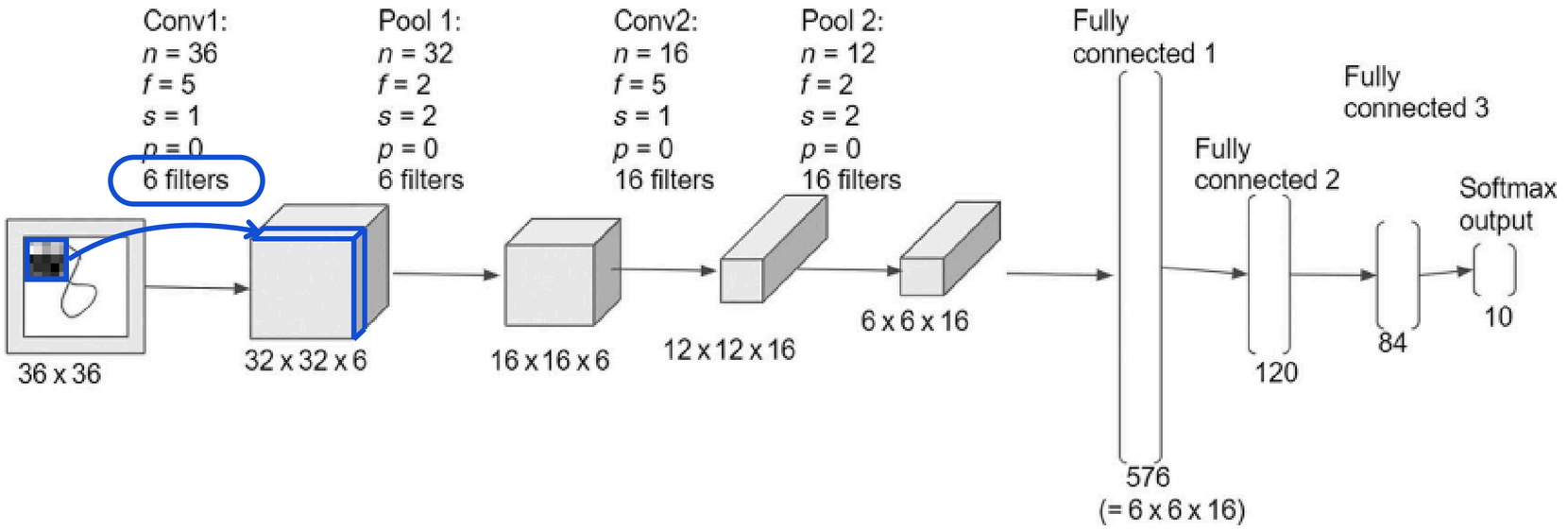


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

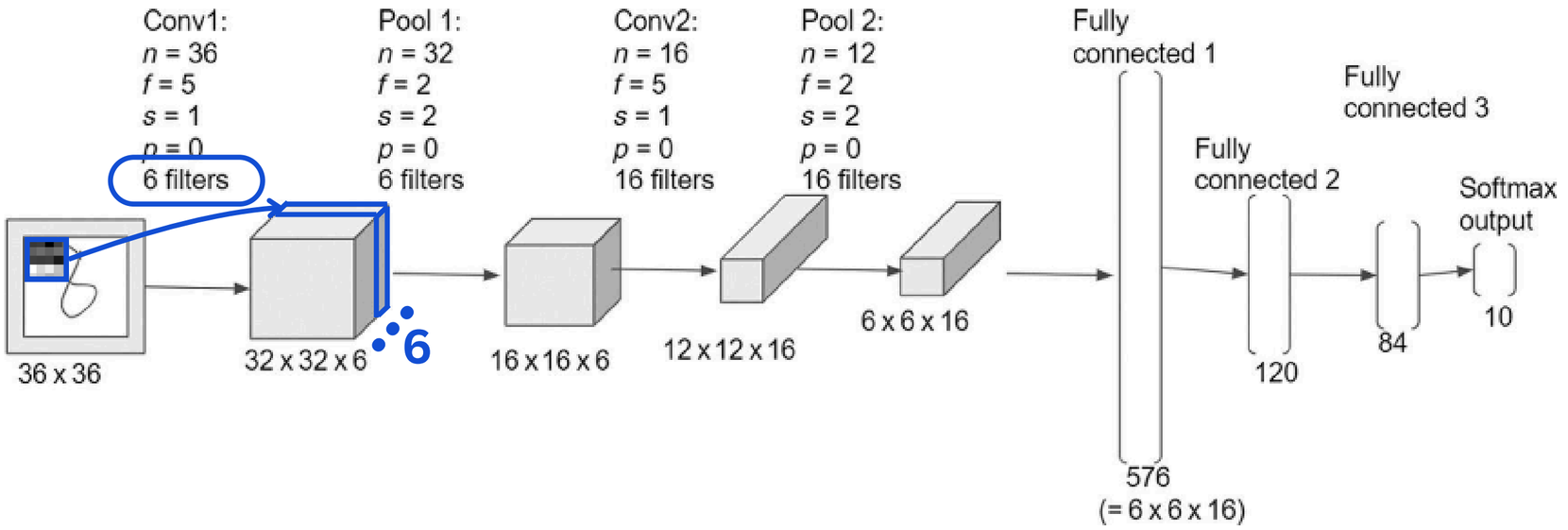


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

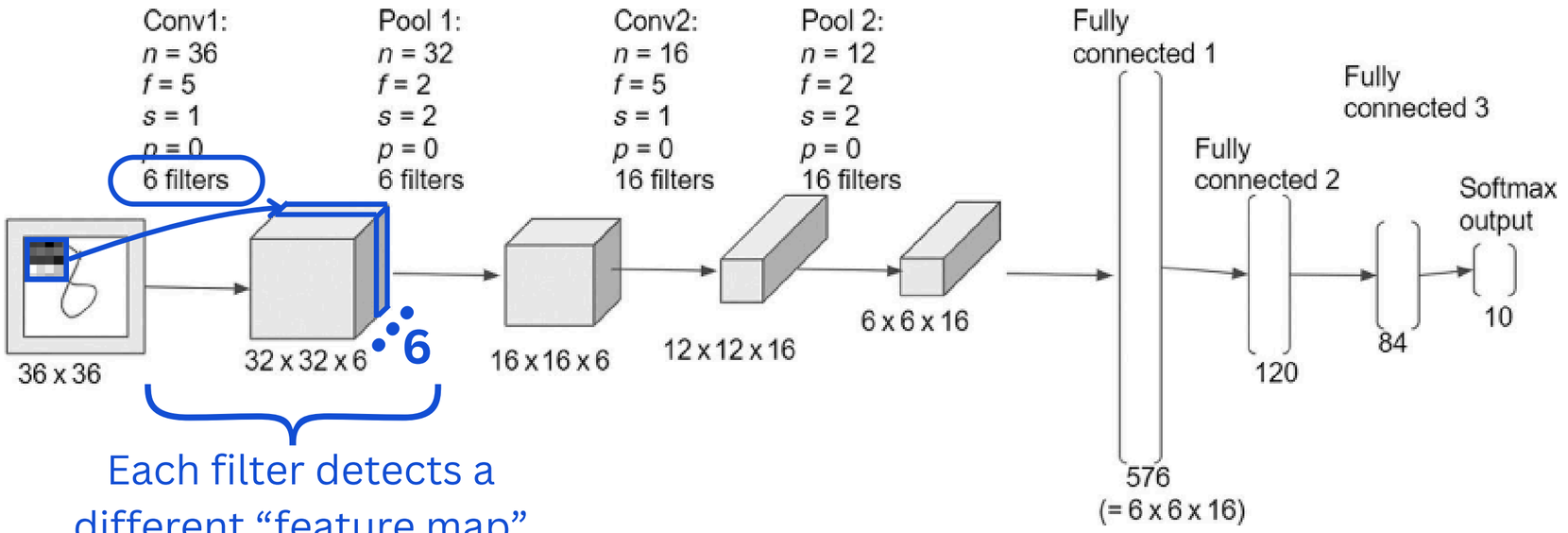


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

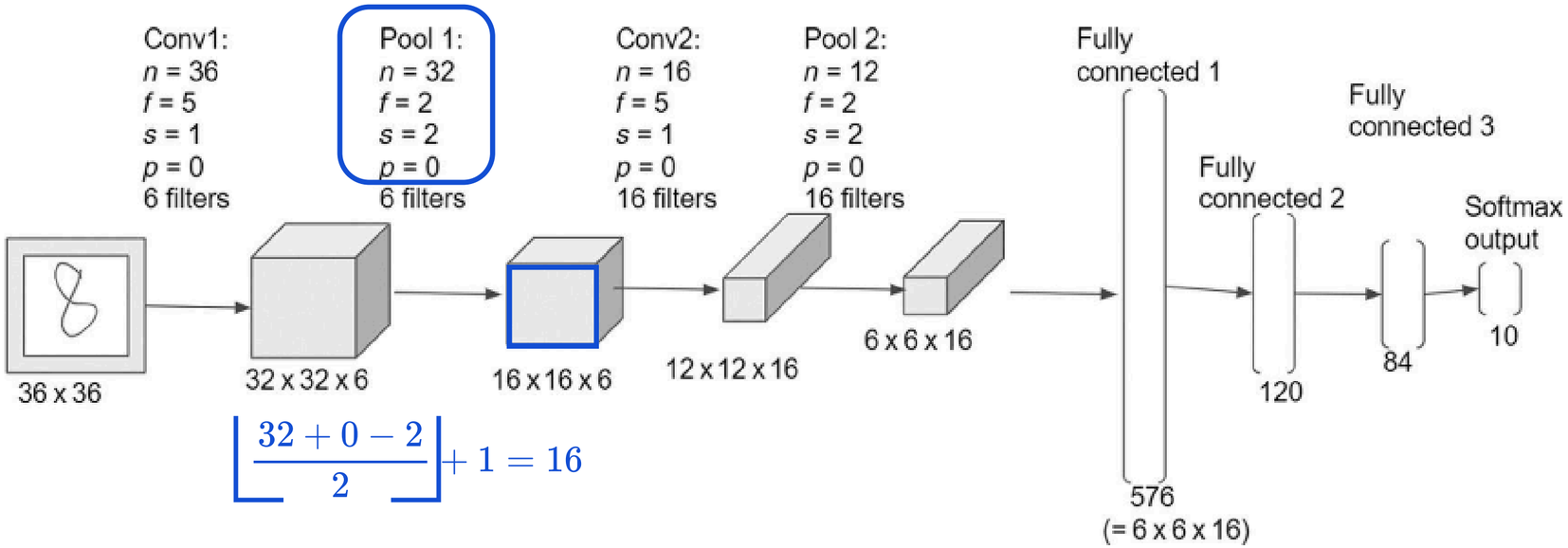


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

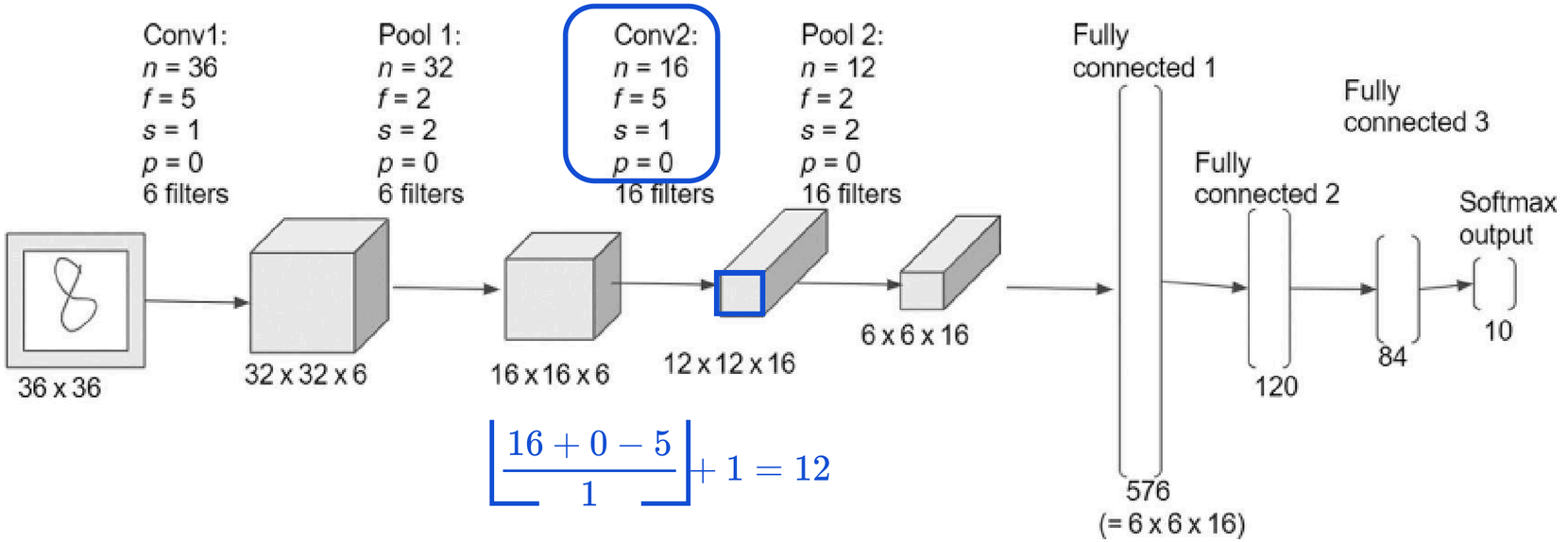
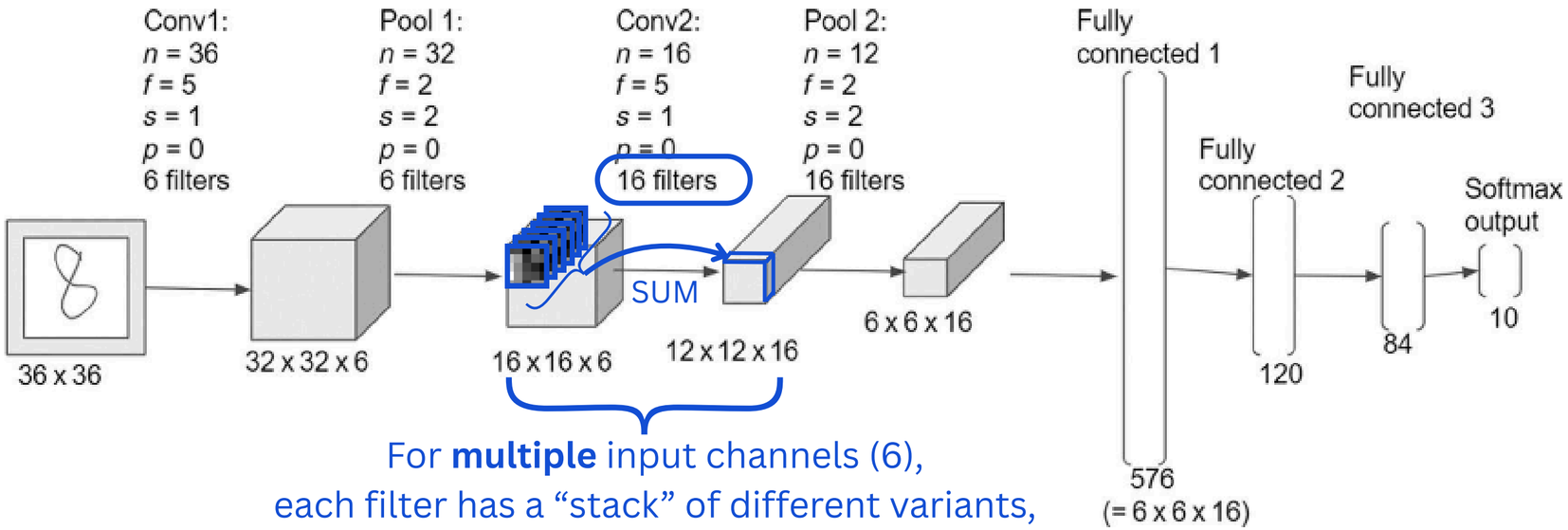


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)



For **multiple** input channels (6),
each filter has a "stack" of different variants,
one per-input channel
→ **SUM convolutions** at each sliding window

FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

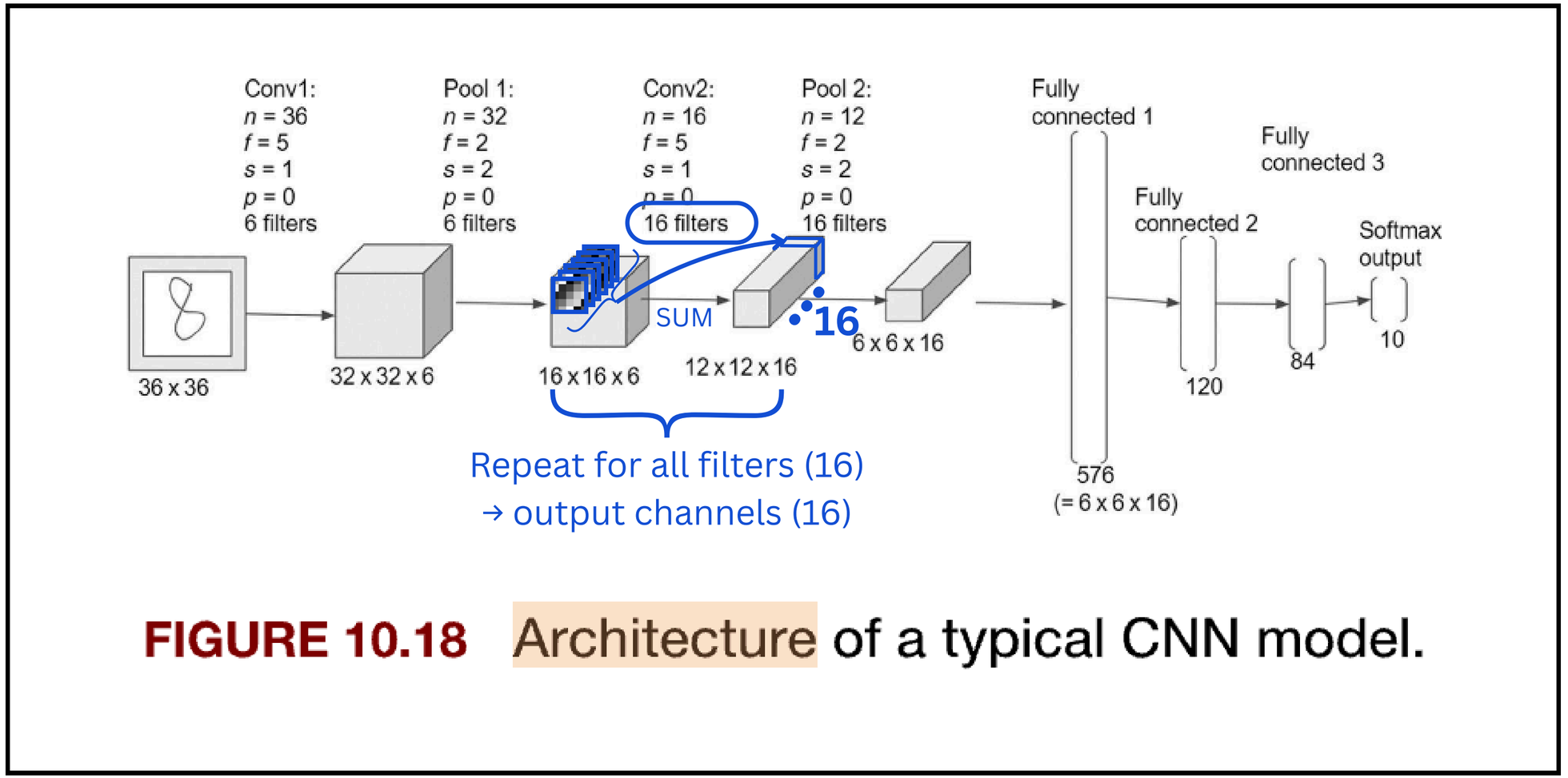


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

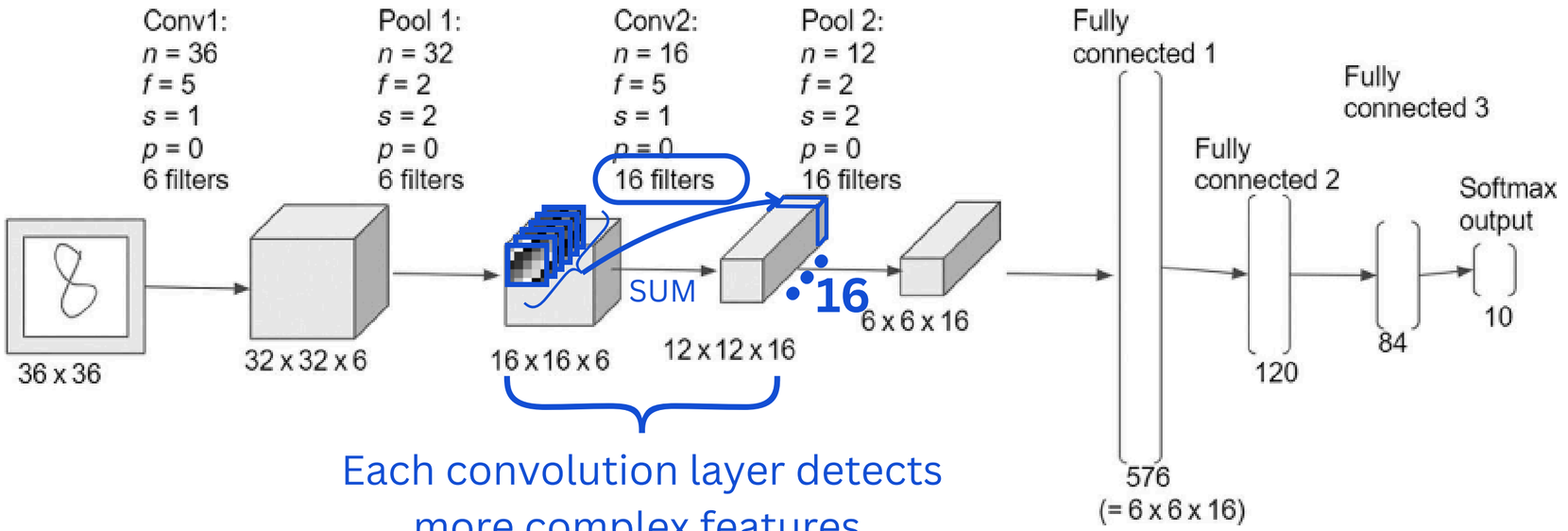


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

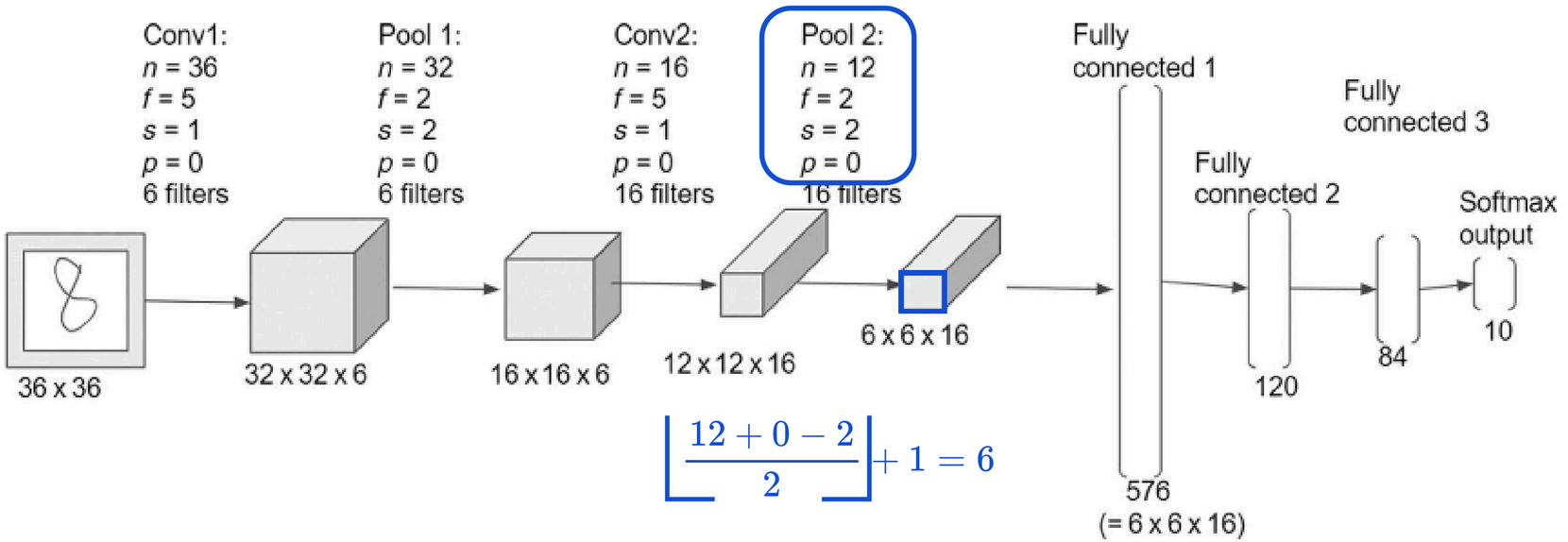


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

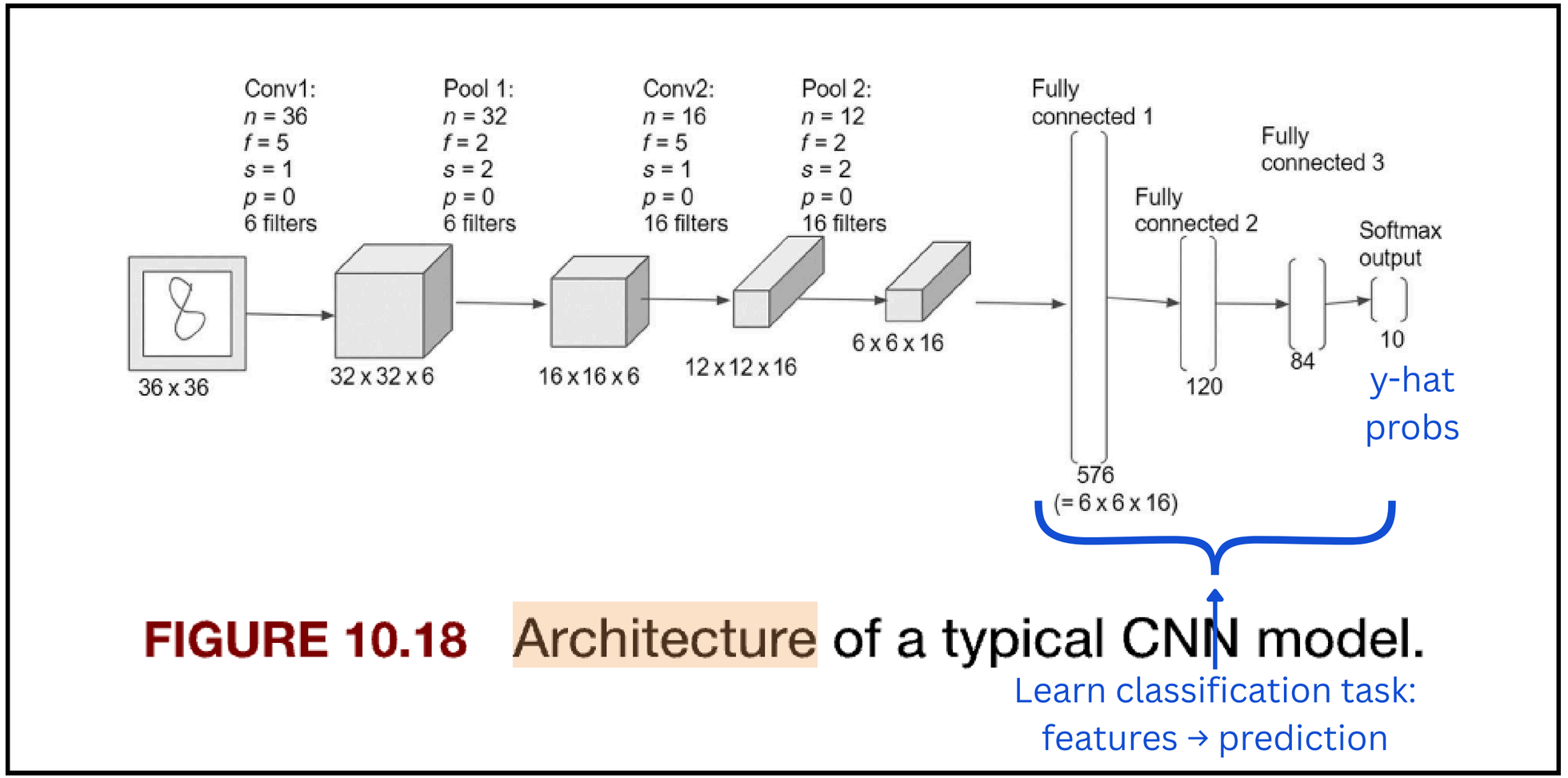
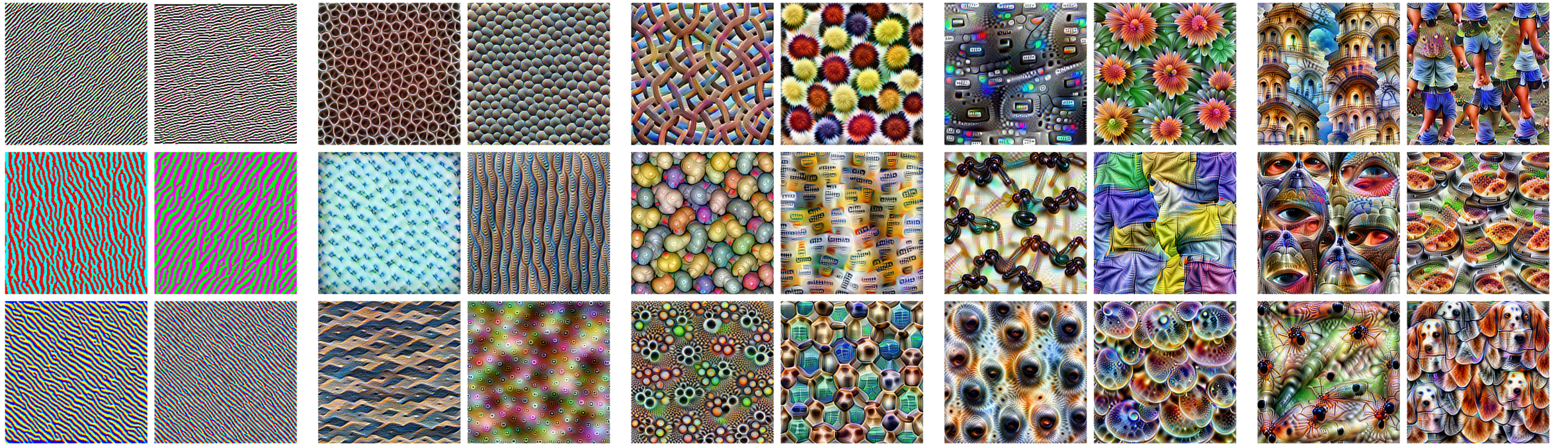


FIGURE 10.18 Architecture of a typical CNN model.

Figure: Kotu & Deshpande, 2019

Convolutional Neural Network (CNN)

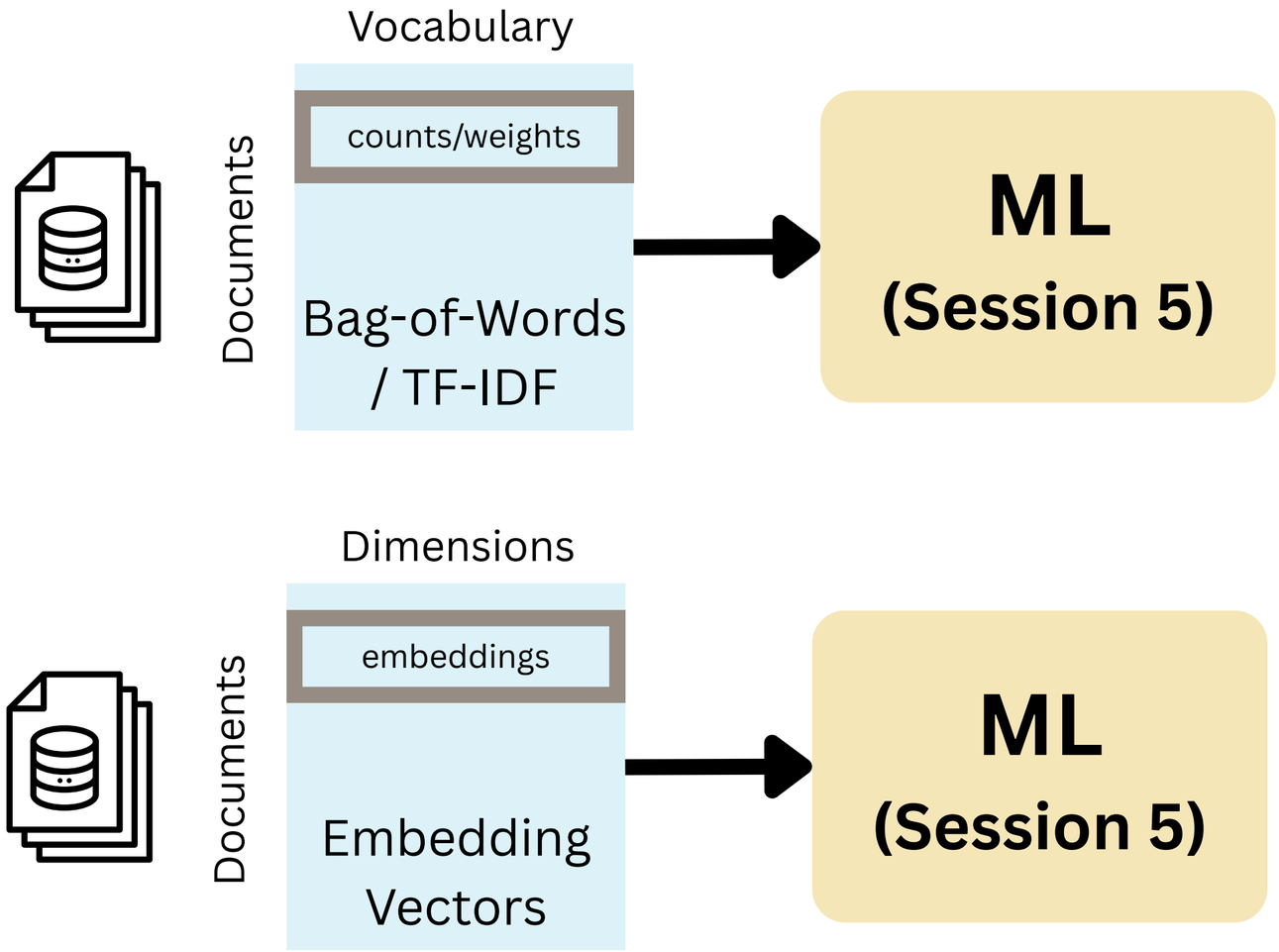


Edges (layer conv2d0) **Textures** (layer mixed3a) **Patterns** (layer mixed4a) **Parts** (layers mixed4b & mixed4c) **Objects** (layers mixed4d & mixed4e)

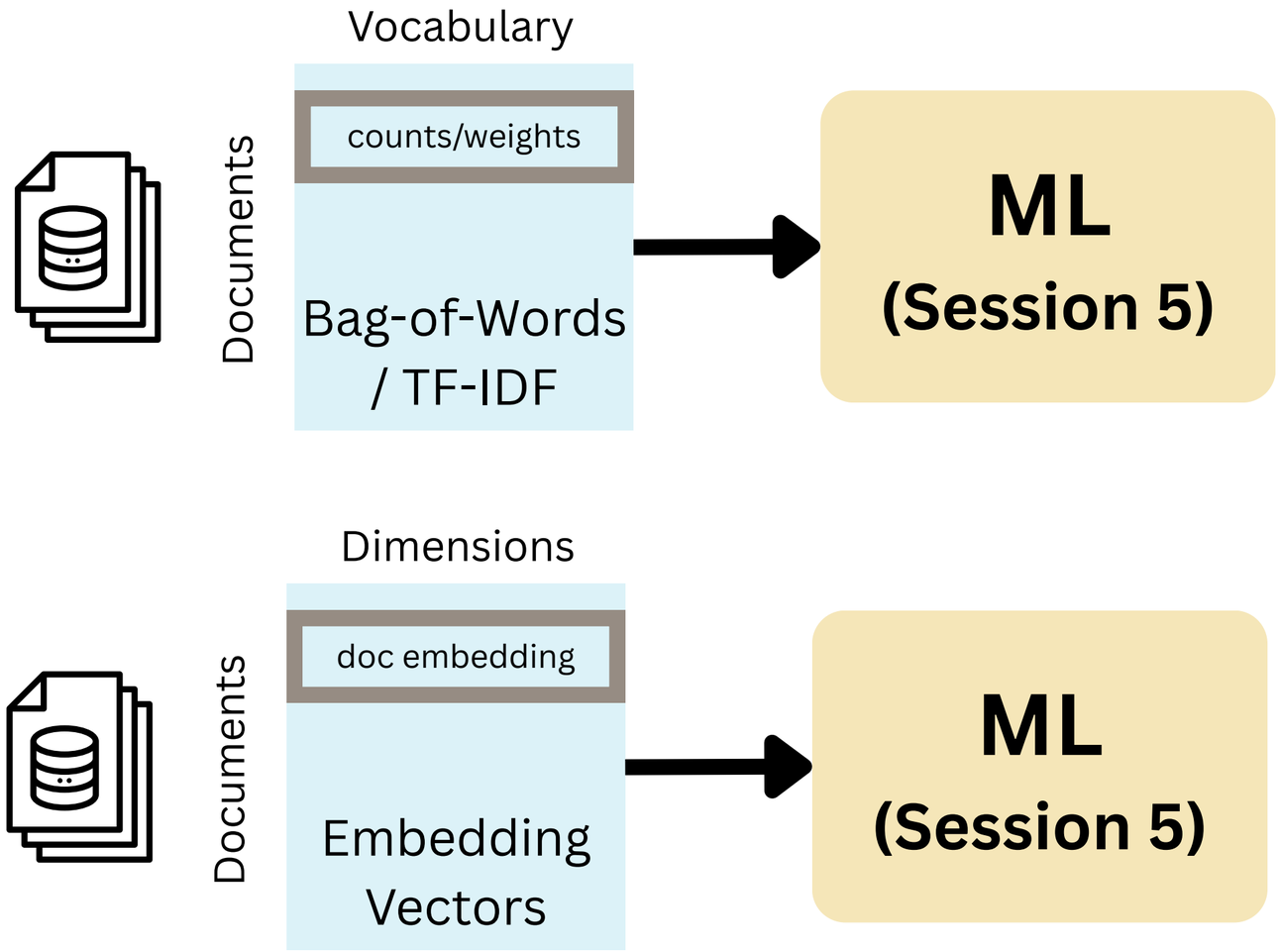
Feature visualization allows us to see how GoogLeNet [1], trained on the ImageNet [2] dataset, builds up its understanding of images over many layers. Visualizations of all channels are available in the appendix.

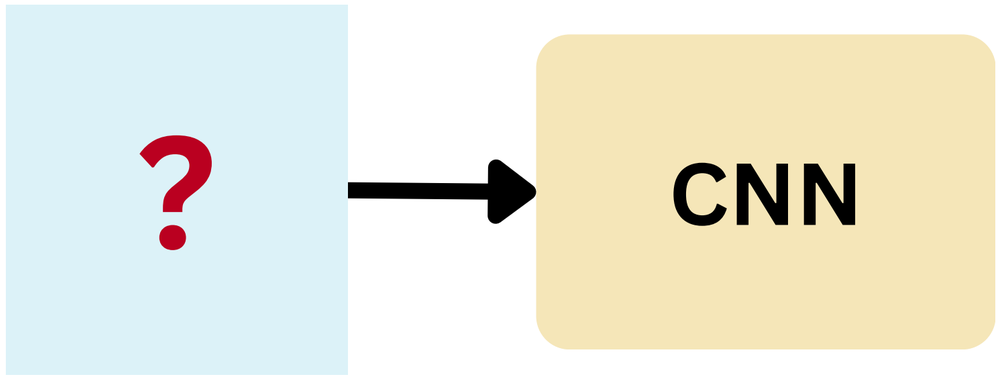
Screenshot: distill, 2017

Text Convolution (1D)

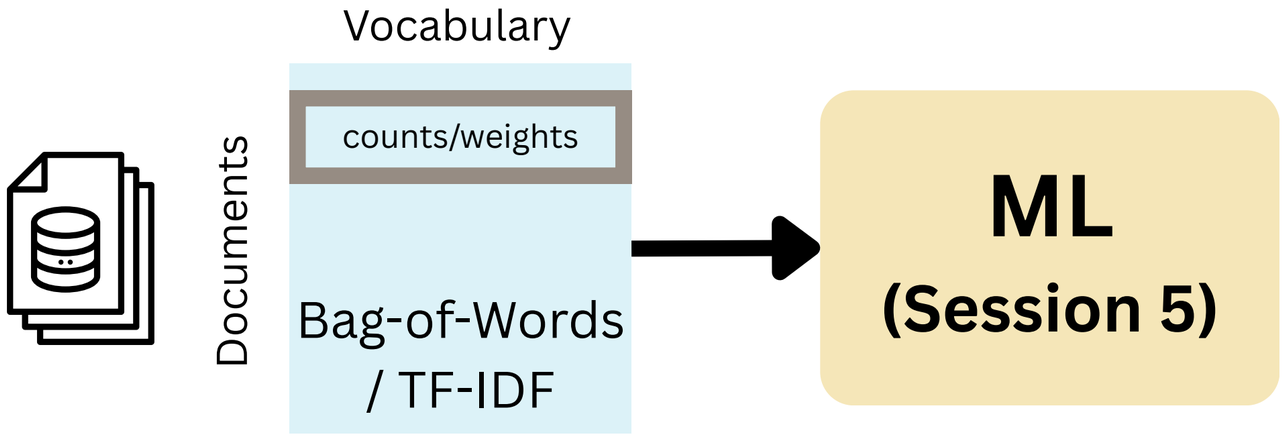


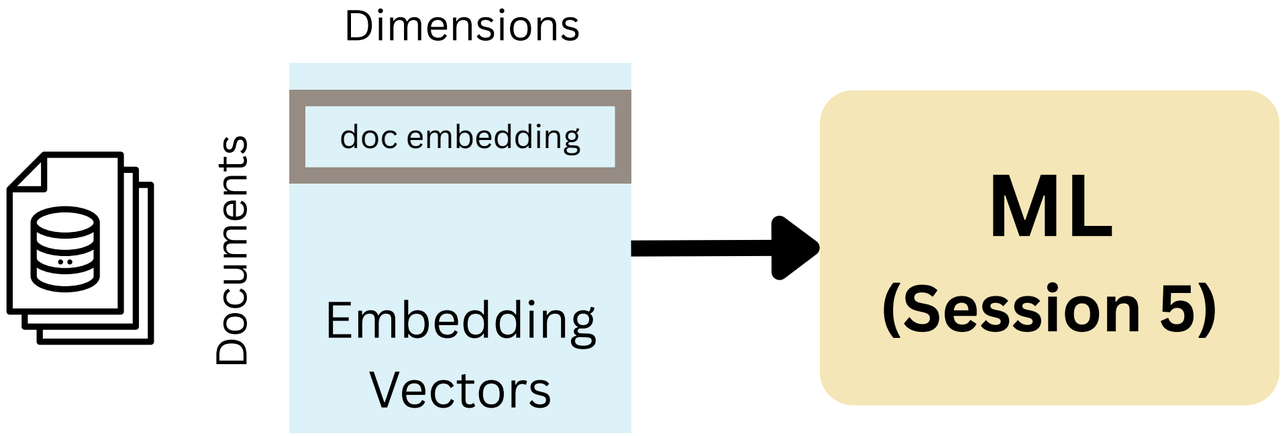
Text Convolution (1D)

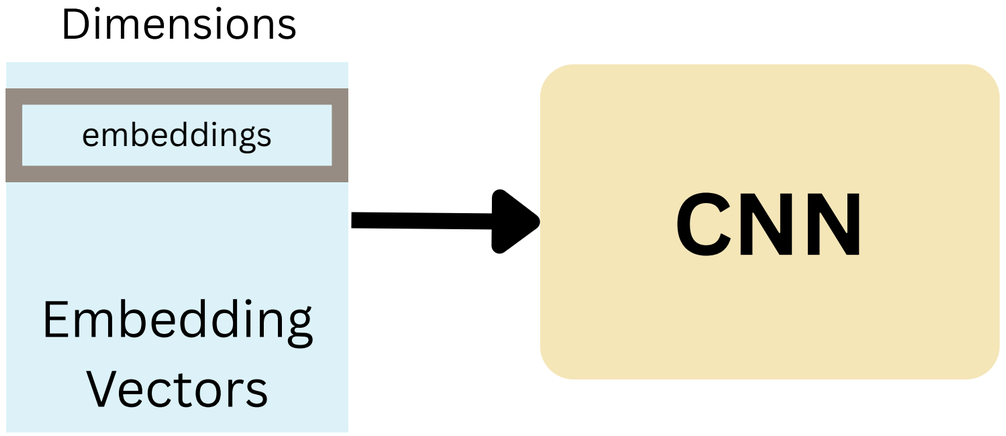




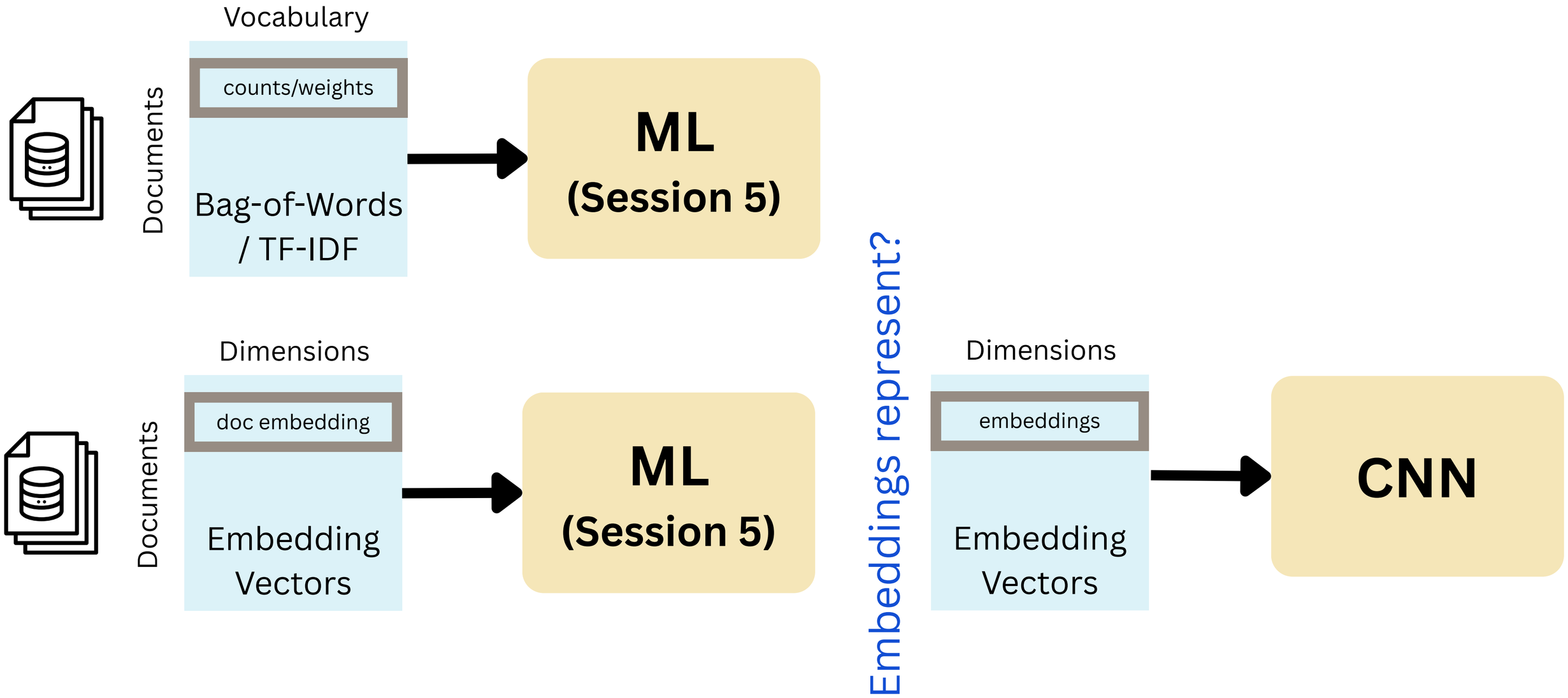
Text Convolution (1D)



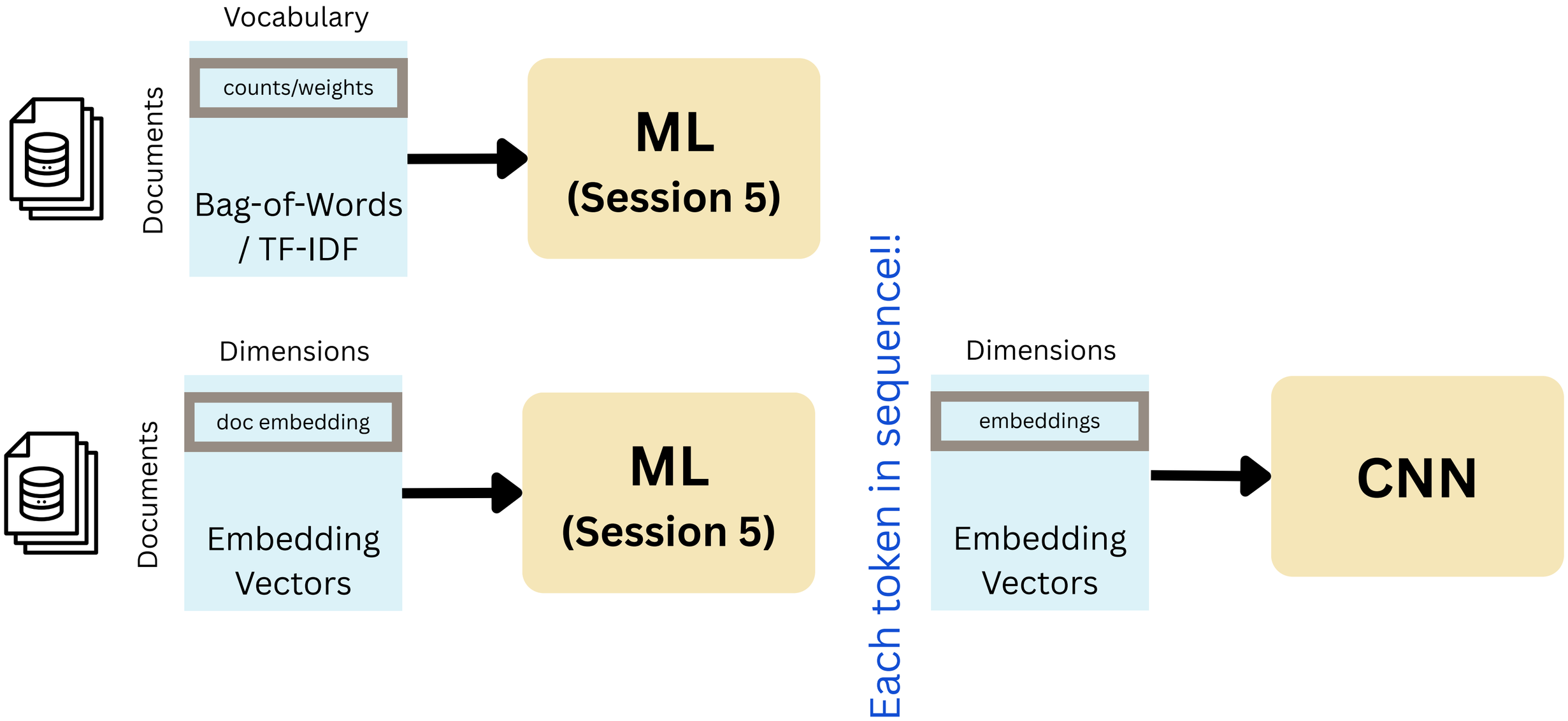




Text Convolution (1D)

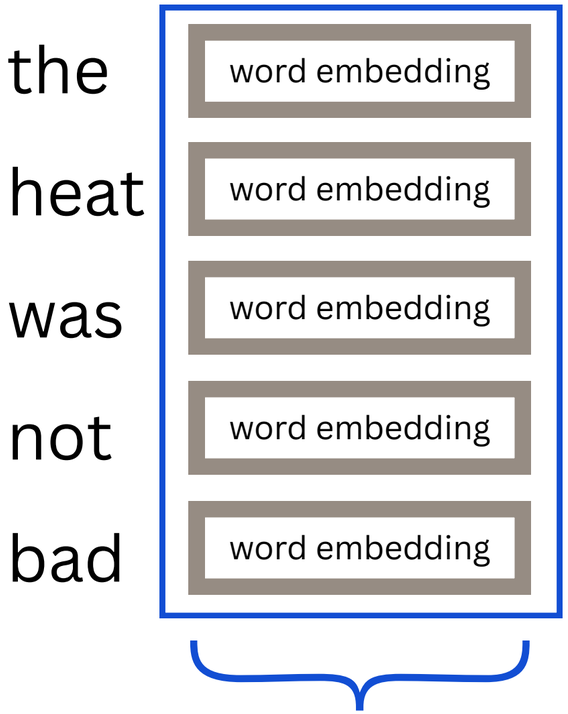


Text Convolution (1D)



Text Convolution (1D)

Input:
5 words x 300 dimensions
(example: Word2Vec 300-dim)



Second input dimension:

- **Embedding vector**
- Like input channels

Text Convolution (1D)

Input:

5 words x 300 dimensions

(example: Word2Vec 300-dim)

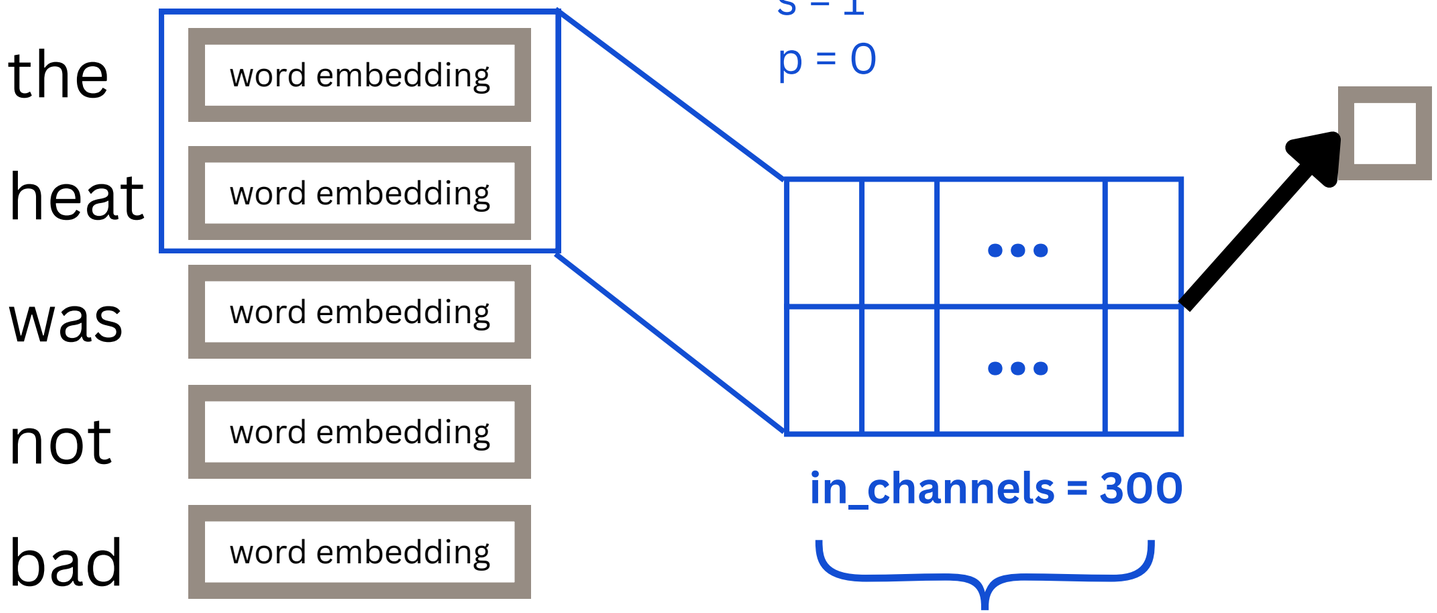
Convolution (1D):

n_in = 5

f = 2

s = 1

p = 0



Filter:

- Kernel size f (words) x
- Input channels (embedding dimensions)

Text Convolution (1D)

Input:

5 words x 300 dimensions

(example: Word2Vec 300-dim)

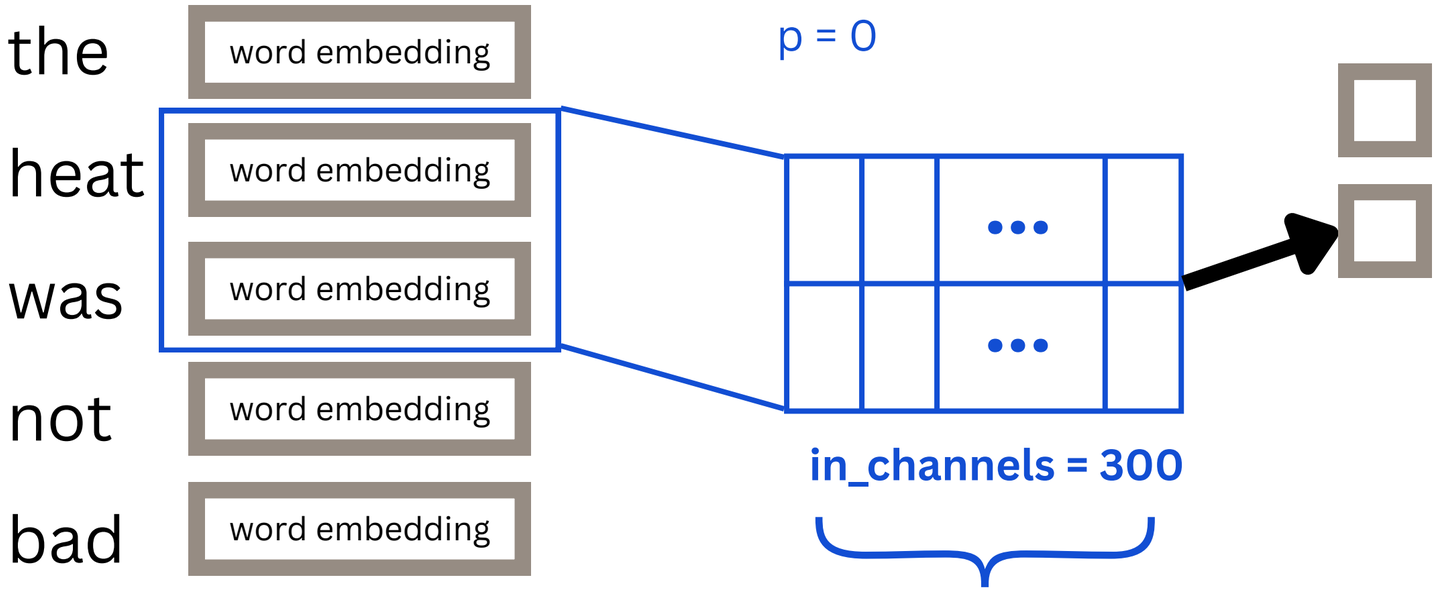
Convolution (1D):

n_in = 5

f = 2

s = 1

p = 0



Filter:

- Kernel size f (words) x
- Input channels (dimensions)
- **Slides across text in ONE dimension (1D)**

Text Convolution (1D)

Input:

5 words x 300 dimensions

(example: Word2Vec 300-dim)

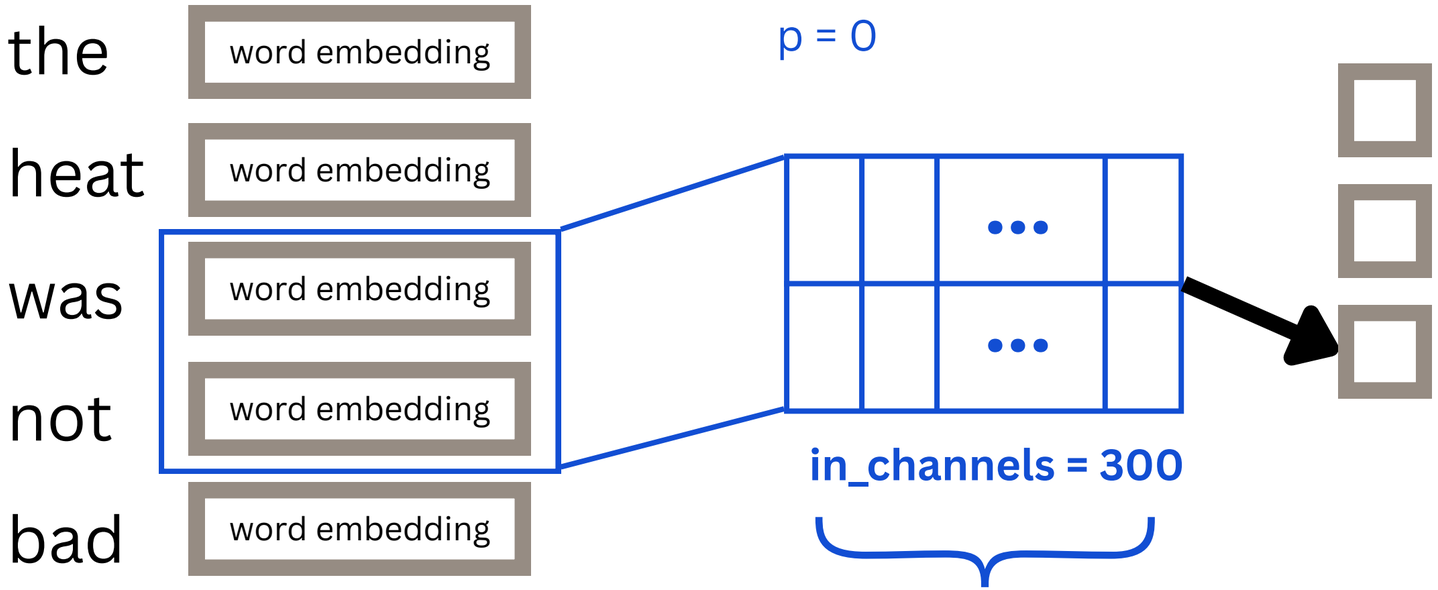
Convolution (1D):

n_in = 5

f = 2

s = 1

p = 0



Filter:

- Kernel size f (words) x
- Input channels (dimensions)
- **Slides across text in ONE dimension (1D)**

Text Convolution (1D)

Input:

5 words x 300 dimensions

(example: Word2Vec 300-dim)

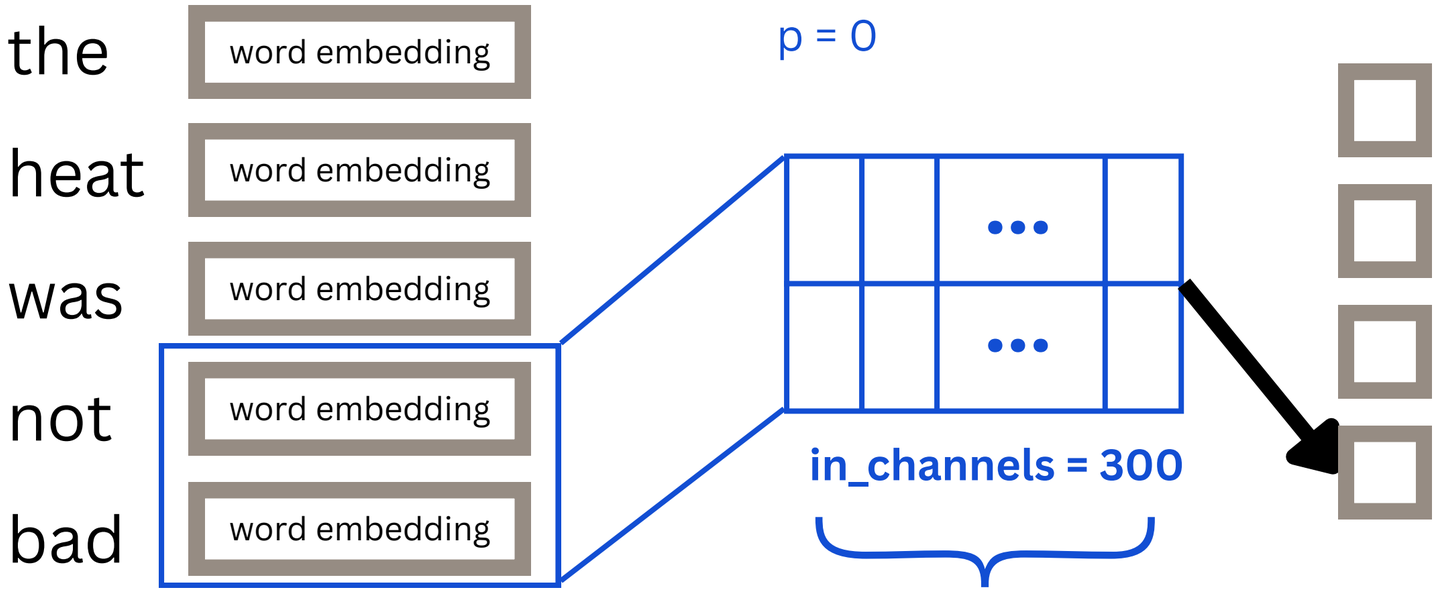
Convolution (1D):

n_in = 5

f = 2

s = 1

p = 0



Filter:

- Kernel size f (words) x
- Input channels (dimensions)
- **Slides across text in ONE dimension (1D)**

Text Convolution (1D)

Input:

5 words x 300 dimensions

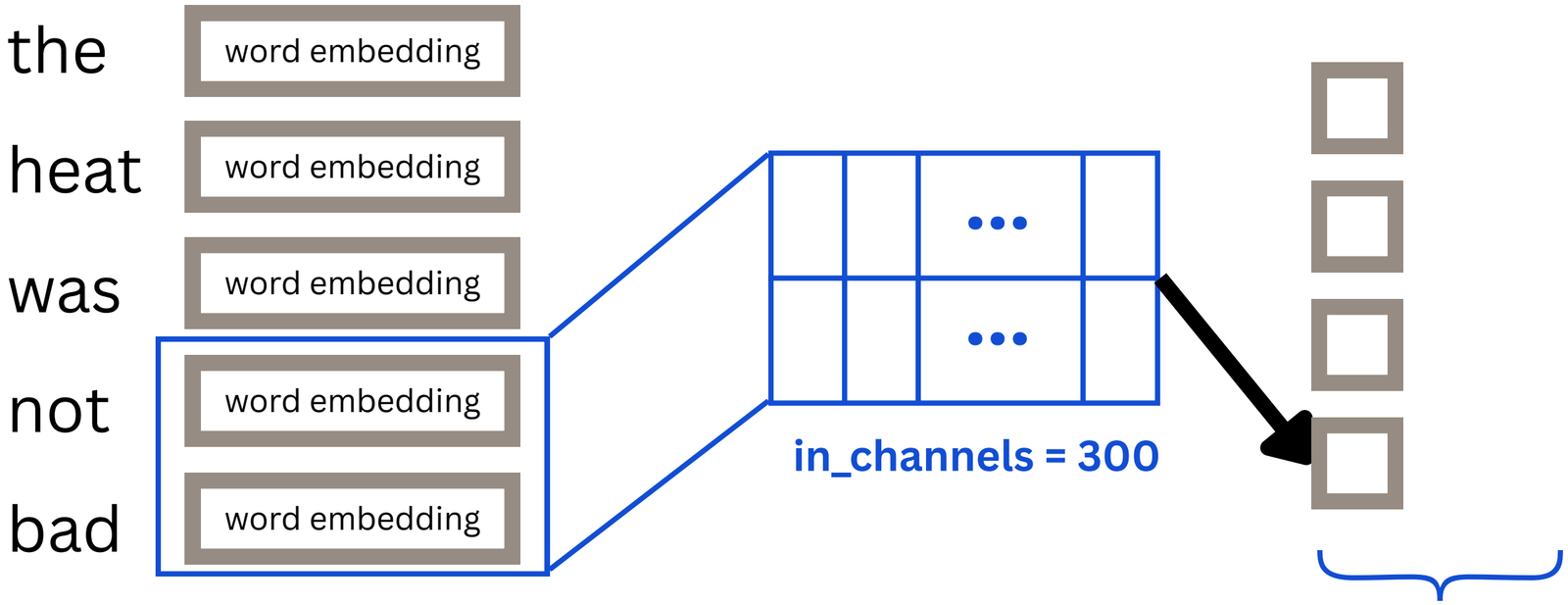
(example: Word2Vec 300-dim)

Convolution (1D):

$$\left\lfloor \frac{5+0-2}{1} \right\rfloor + 1 = 4$$

Conv Output:

n_out = 4



Feature map

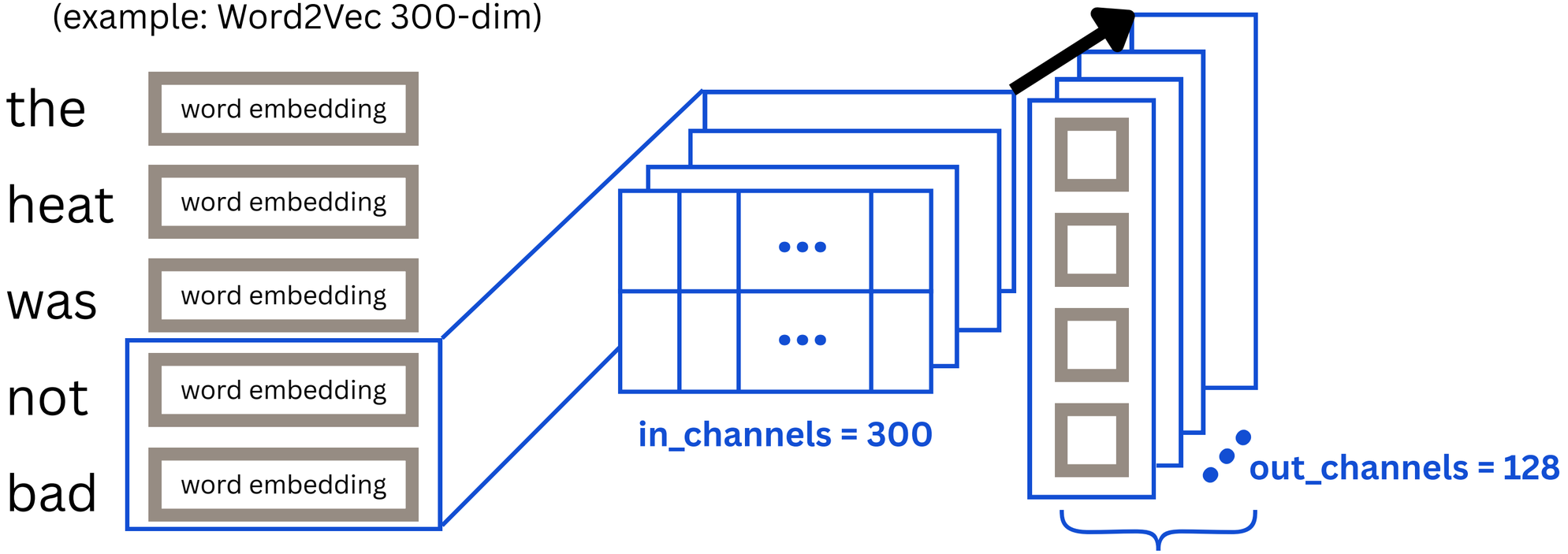
- N-grams (phrases) detected

Text Convolution (1D)

Input:
5 words x 300 dimensions
(example: Word2Vec 300-dim)

Convolution (1D):

Conv Output:
4 x 128 out_ch



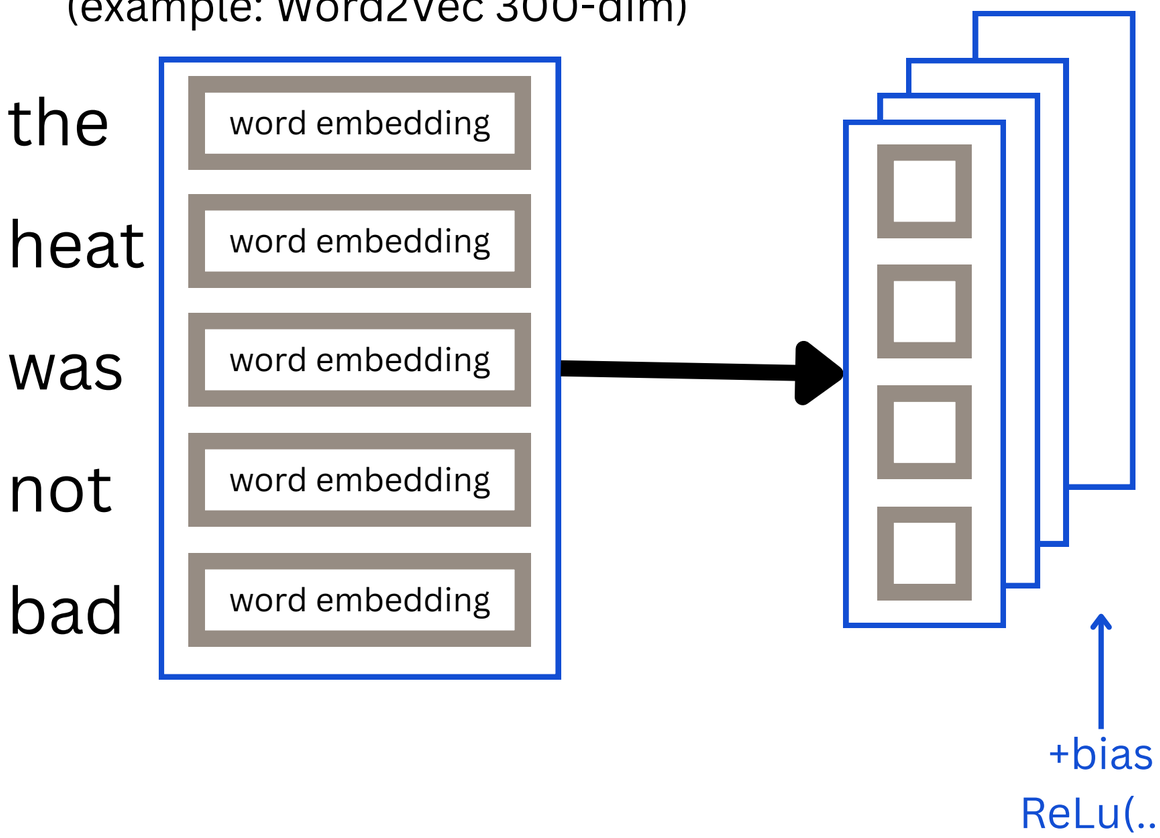
Feature maps

- N-grams (phrases) of different families of meanings detected

Reference: Jacovi et al., 2018

Text Convolution (1D)

Input:
5 words x 300 dimensions
(example: Word2Vec 300-dim)



Conv Output:
4 x 128 out_ch

+bias
ReLu(...)

Max pooling:

n_in = 4
f = 4
s = 1
p = 0

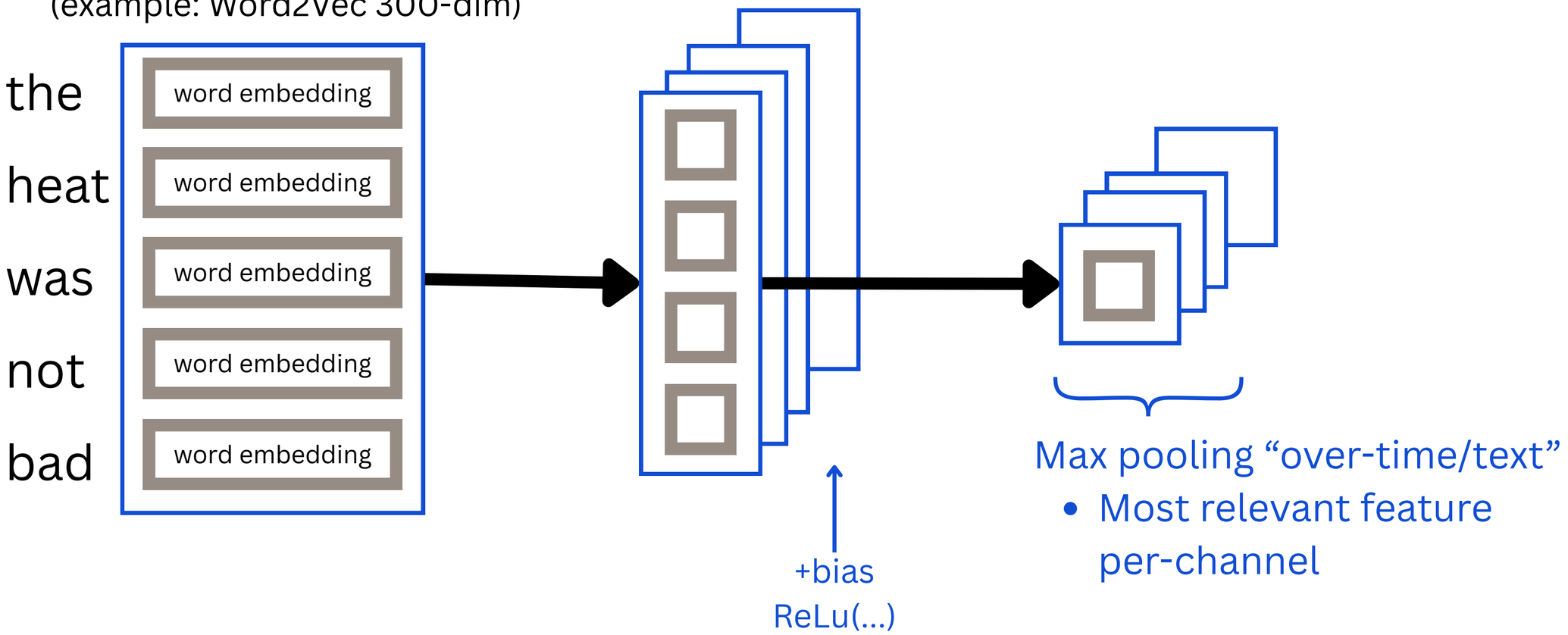
$$\left\lfloor \frac{4 + 0 - 4}{1} \right\rfloor + 1 = 1$$

Text Convolution (1D)

Input:
5 words x 300 dimensions
(example: Word2Vec 300-dim)

Conv Output:
4 x 128 out_ch

Pooling output:
1 x 128 out_ch

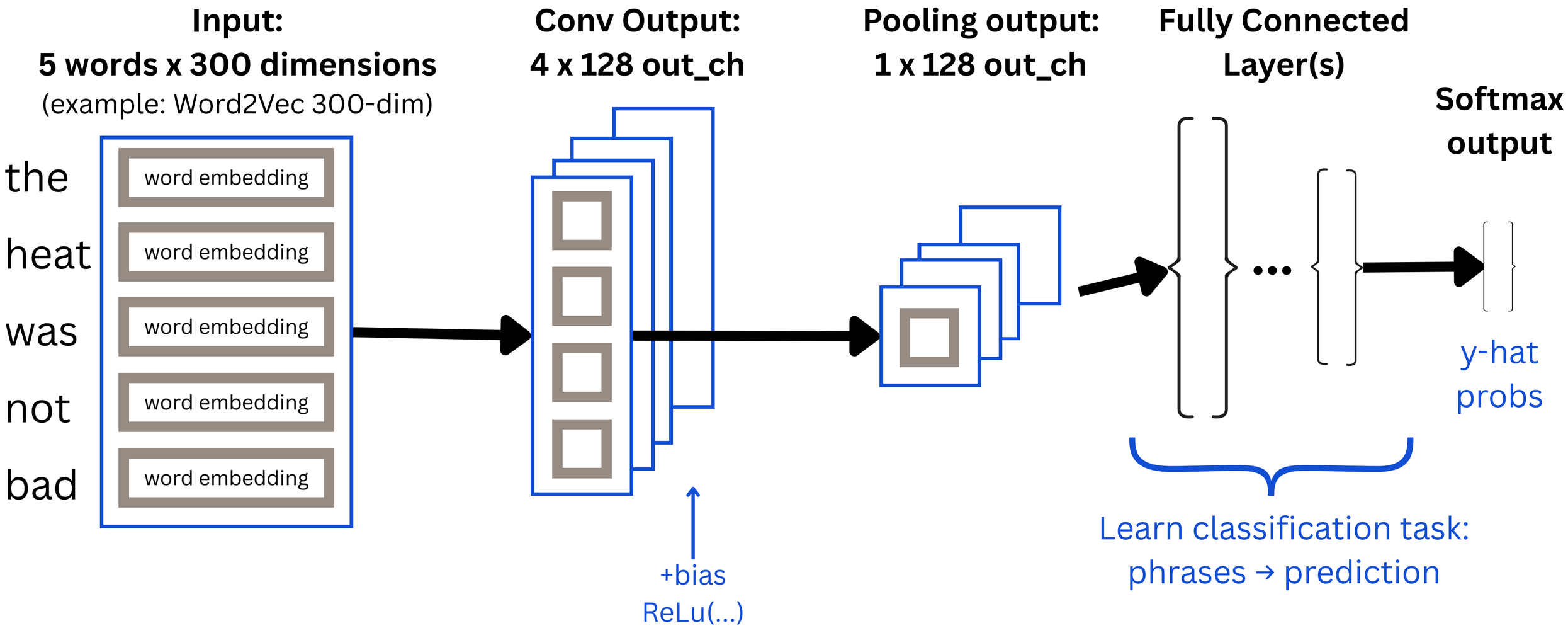


Max pooling “over-time/text”

- Most relevant feature per-channel

Reference: Jacovi et al., 2018

Text Convolution (1D)

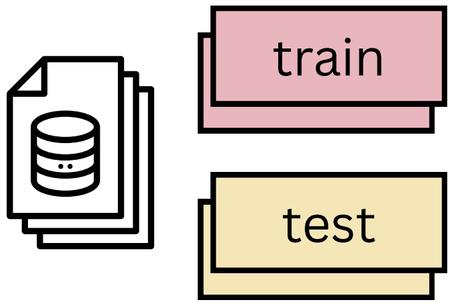


PyTorch: Data and Embeddings



PyTorch: Data and Embeddings

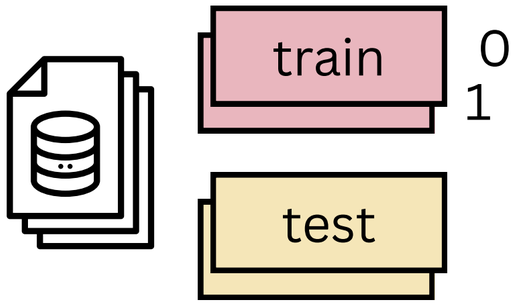
1. Prepare




```
df_train, df_test =
train_test_split(...)
```

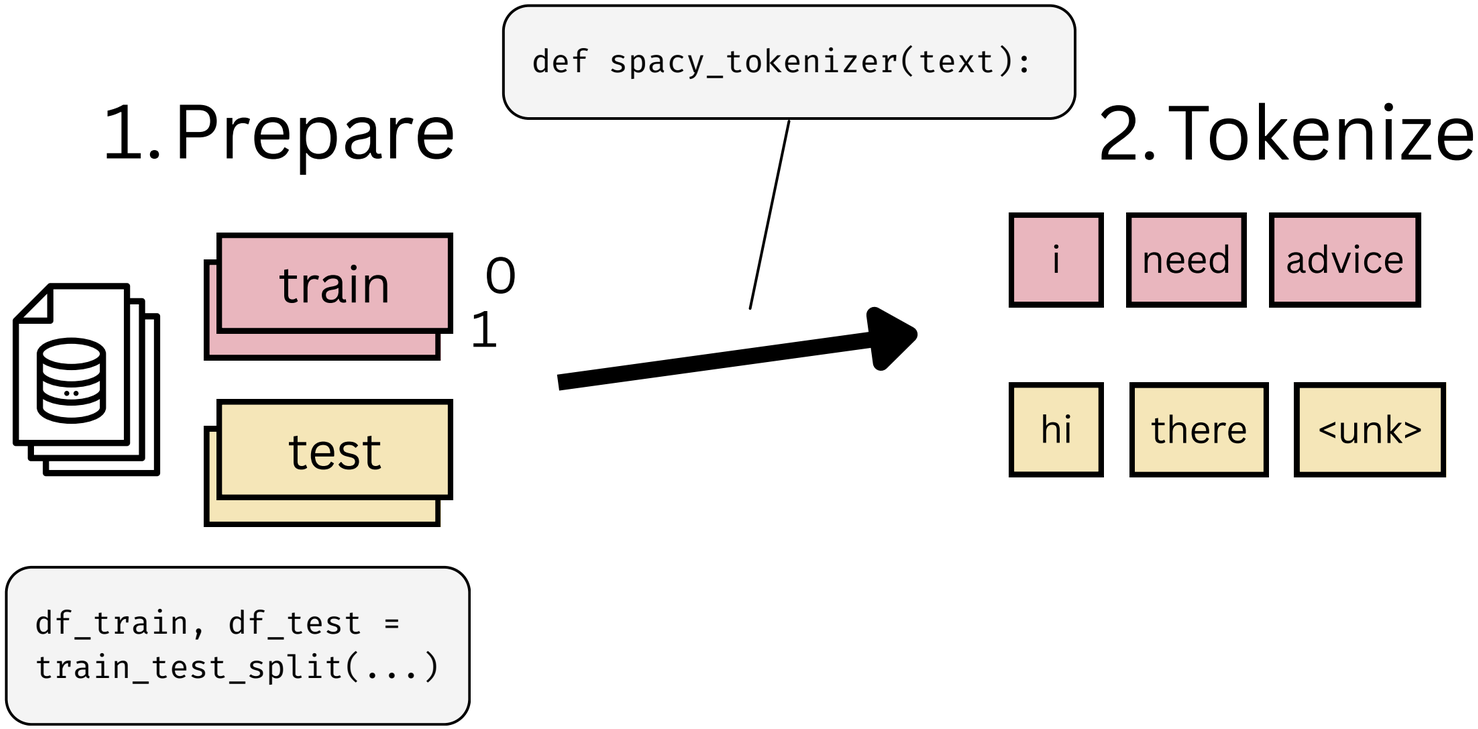
PyTorch: Data and Embeddings

1. Prepare

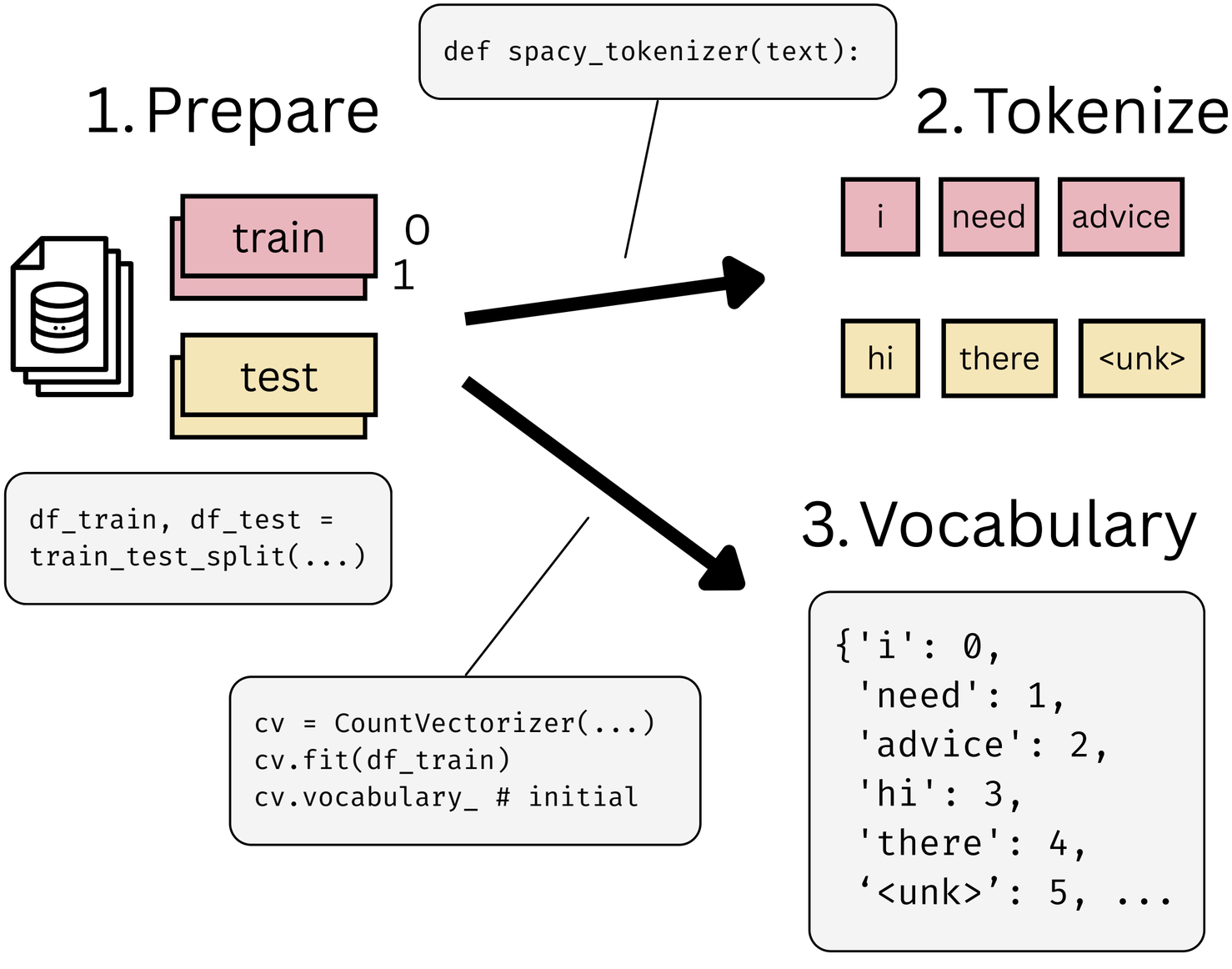



```
df_train, df_test =
train_test_split(...)
```

PyTorch: Data and Embeddings



PyTorch: Data and Embeddings



PyTorch: Data and Embeddings

```
def collate_fn(data):
    [vocab[token] for ...
```

```
def spacy_tokenizer(text):
```

1. Prepare



train 0
1

test

```
df_train, df_test =
train_test_split(...)
```

2. Tokenize

i | need | advice

hi | there | <unk>

3. Vocabulary

```
cv = CountVectorizer(...)
cv.fit(df_train)
cv.vocabulary_ # initial
```

```
{'i': 0,
 'need': 1,
 'advice': 2,
 'hi': 3,
 'there': 4,
 '<unk>': 5, ...
```

4. Encode

0 | 1 | 2

3 | 4 | 5

PyTorch: Data and Embeddings

```
def collate_fn(data):
    [vocab[token] for ...
```

```
def spacy_tokenizer(text):
```

1. Prepare



train 0
1

test

```
df_train, df_test =
train_test_split(...)
```

2. Tokenize

i | need | advice

hi | there | <unk>

3. Vocabulary

```
cv = CountVectorizer(...)
cv.fit(df_train)
cv.vocabulary_ # initial
```

```
{'i': 0,
 'need': 1,
 'advice': 2,
 'hi': 3,
 'there': 4,
 '<unk>': 5, ...
```

4. Encode

0 | 1 | 2 | ••• | 5 | 7

3 | 4 | 5

PyTorch: Data and Embeddings

```
def collate_fn(data):
    [vocab[token] for ...
    ...[:max_seq_length]
```

```
def spacy_tokenizer(text):
```

1. Prepare



train 0
1

test

```
df_train, df_test =
train_test_split(...)
```

2. Tokenize

i | need | advice

hi | there | <unk>

4. Encode, truncate

0 | 1 | 2 | ••• | 12 | 7

3 | 4 | 5

3. Vocabulary

```
cv = CountVectorizer(...)
cv.fit(df_train)
cv.vocabulary_ # initial
```

```
{'i': 0,
 'need': 1,
 'advice': 2,
 'hi': 3,
 'there': 4,
 '<unk>': 5, ...
```

PyTorch: Data and Embeddings

```
def collate_fn(data):
    [vocab[token] for ...
    ...[:max_seq_length]
    pad_sequence(...)
```

```
def spacy_tokenizer(text):
```

1. Prepare



train 0
1

test

```
df_train, df_test =
train_test_split(...)
```

2. Tokenize

i | need | advice

hi | there | <unk>

3. Vocabulary

```
cv = CountVectorizer(...)
cv.fit(df_train)
cv.vocabulary_ # initial
```

```
{'<pad>': 0,
 'i': 1,
 'need': 2,
 'advice': 3,
 'hi': 4,
 'there': 5,
 '<unk>': 6, ...
```

4. Encode, trun., pad

1 | 2 | 3 | ••• | 12 | 7

4 | 5 | 6 | ••• | 0

PyTorch: Data and Embeddings

```
def collate_fn(data):
    [vocab[token] for ...
    ...[:max_seq_length]
    pad_sequence(...)
```

```
def spacy_tokenizer(text):
```

1. Prepare



train 0
test 1

```
df_train, df_test =
train_test_split(...)
```

```
cv = CountVectorizer(...)
cv.fit(df_train)
cv.vocabulary_ # initial
```

2. Tokenize

i need advice

hi there <unk>

3. Vocabulary

```
{'<pad>': 0,
 'i': 1,
 'need': 2,
 'advice': 3,
 'hi': 4,
 'there': 5,
 '<unk>': 6, ...
```

4. Encode, trun., pad

1 2 3 ••• 13 8

4 5 6 ••• 0

5. Embedding Matrix

300 dimensions

[2] word embedding

pre-trained word embeddings

PyTorch: Data and Embeddings

```
def collate_fn(data):
    [vocab[token] for ...
    ...[:max_seq_length]
    pad_sequence(...)
```

```
def spacy_tokenizer(text):
```

1. Prepare



train 0
1

test

```
df_train, df_test =
train_test_split(...)
```

2. Tokenize

i | need | advice

hi | there | <unk>

4. Encode, trun., pad

1 | 2 | 3 ••• 13 | 8

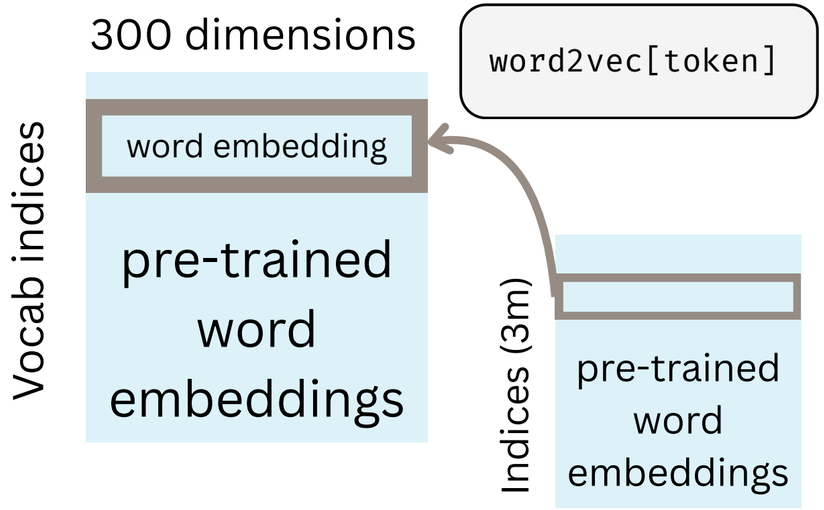
4 | 5 | 6 ••• 0

3. Vocabulary

```
cv = CountVectorizer(...)
cv.fit(df_train)
cv.vocabulary_ # initial
```

```
{'<pad>': 0,
 'i': 1,
 'need': 2,
 'advice': 3,
 'hi': 4,
 'there': 5,
 '<unk>': 6, ...
```

5. Embedding Matrix



PyTorch: Model Layers

```
classCNNModel(nn.Module):
    def __init__
```

nn.Embedding

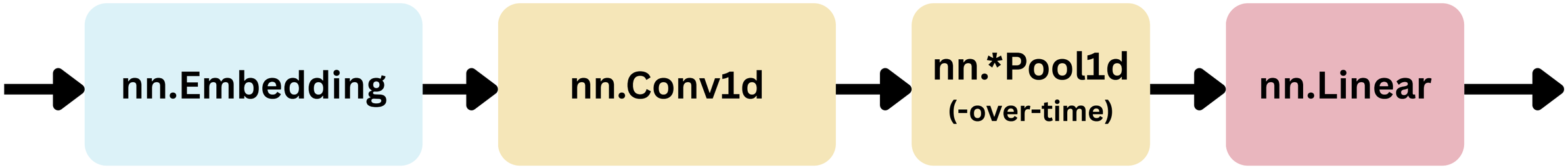
nn.Conv1d

nn.*Pool1d
(-over-time)

nn.Linear

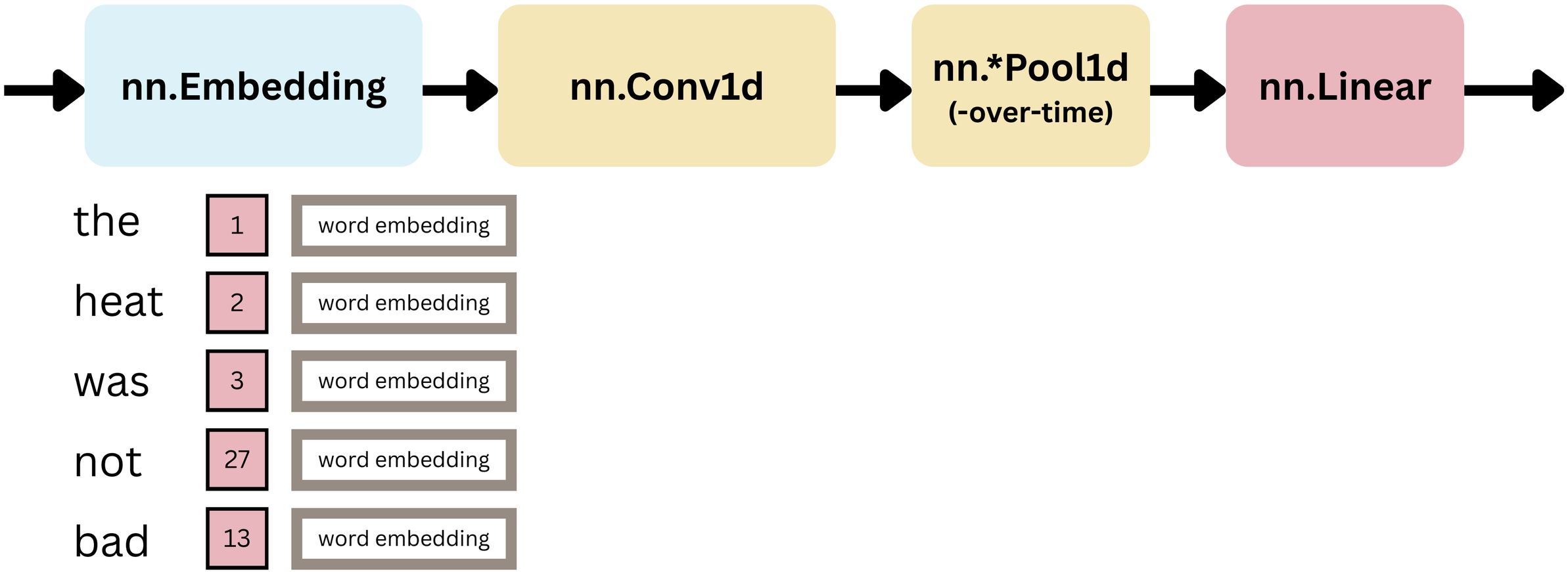
PyTorch: Model Data Flow

```
classCNNModel(nn.Module):
    def forward
```



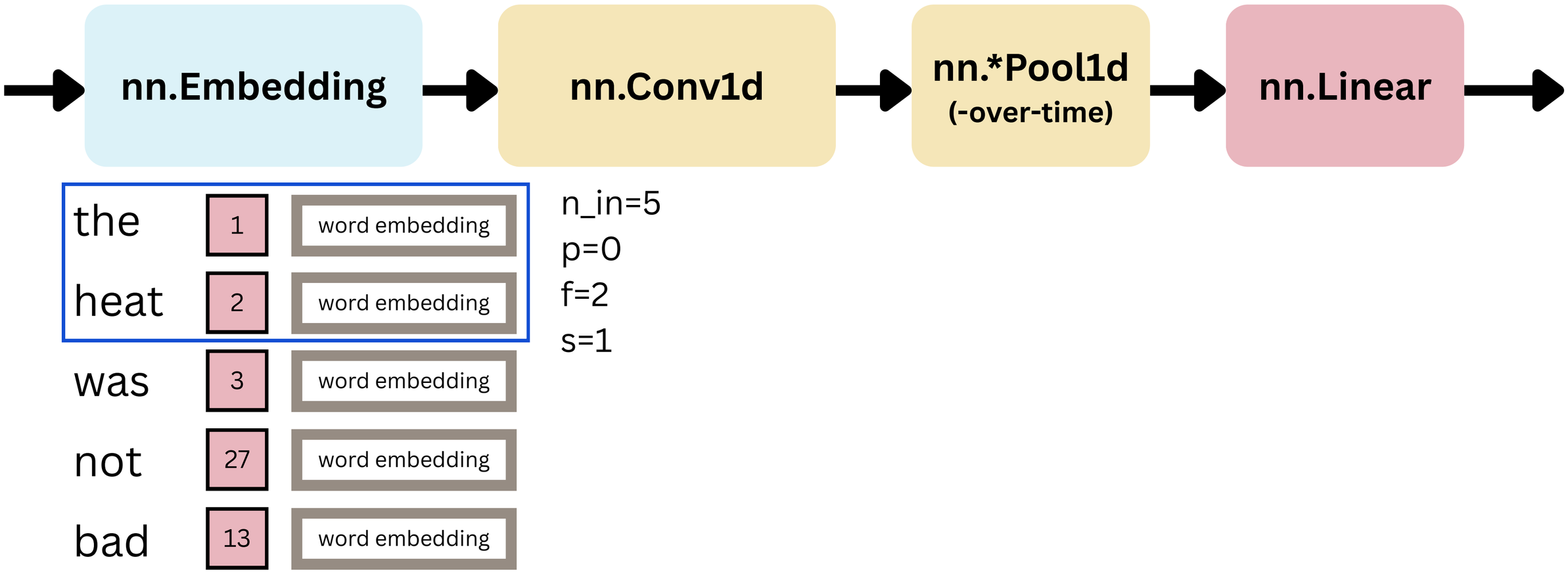
PyTorch: Train/Test Forward Pass

```
def train(...): # or def test
    pred = model(x_batch)
```



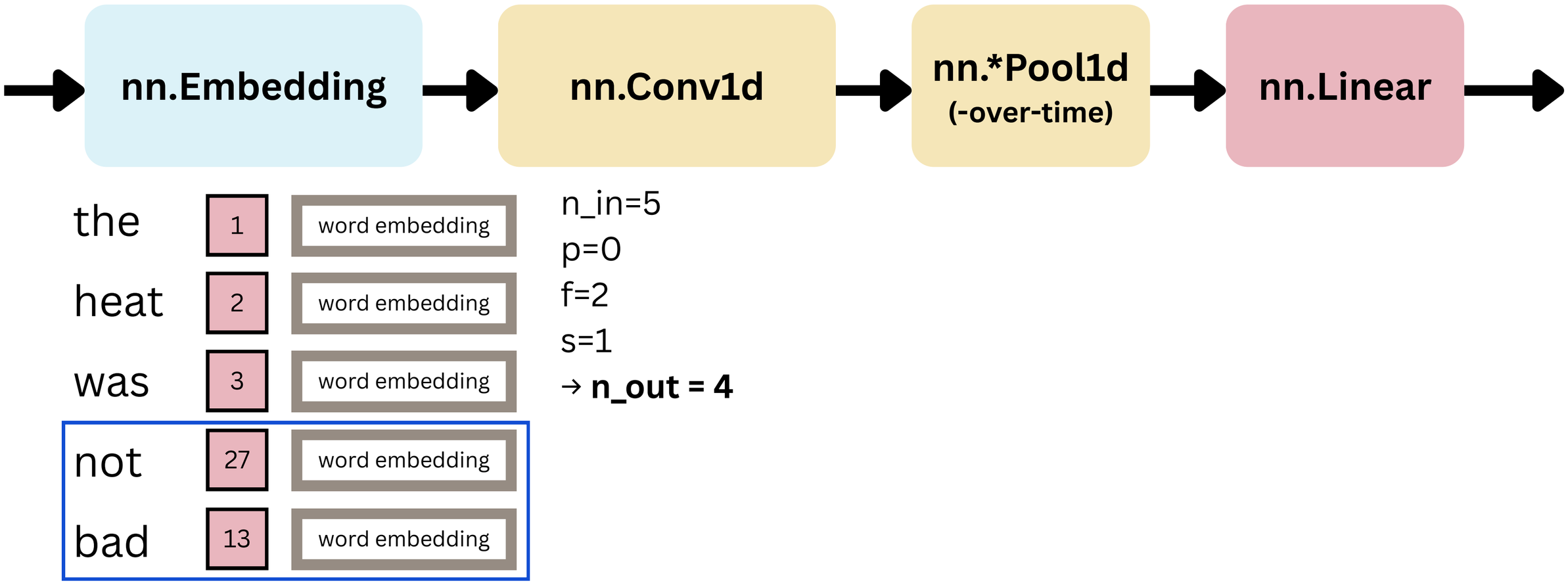
PyTorch: Train/Test Forward Pass

```
def train(...): # or def test
    pred = model(x_batch)
```



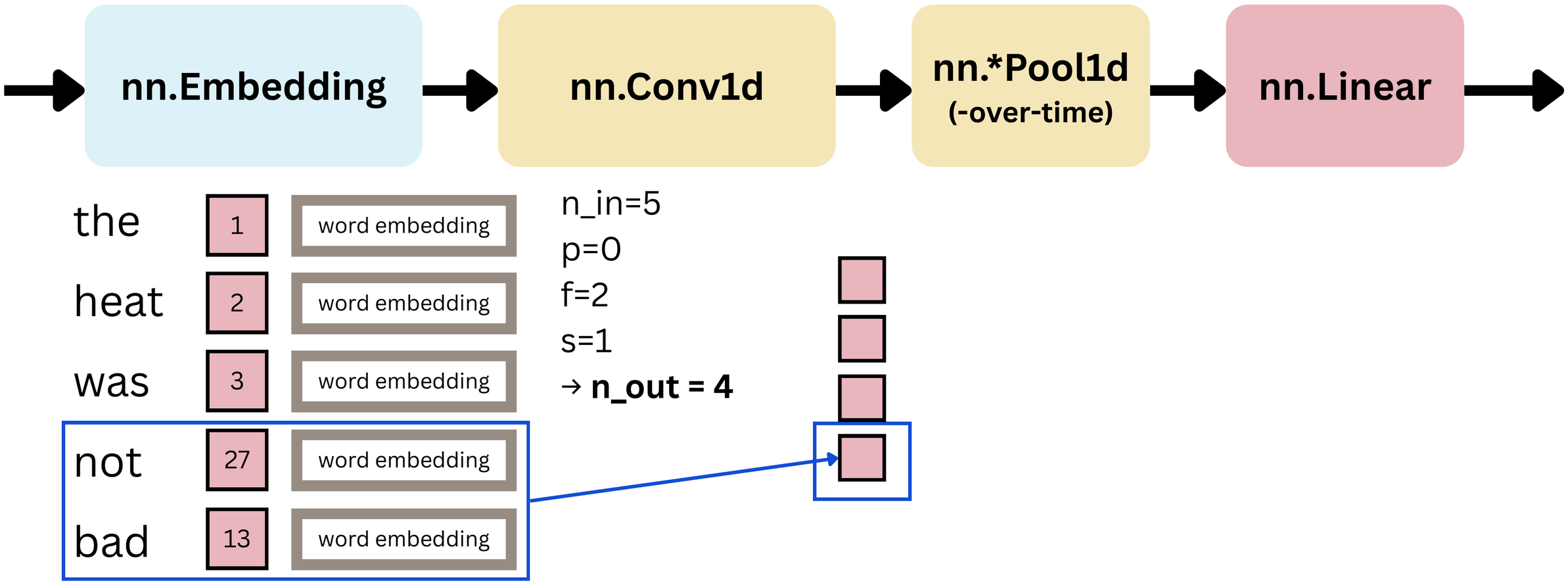
PyTorch: Train/Test Forward Pass

```
def train(...): # or def test
    pred = model(x_batch)
```



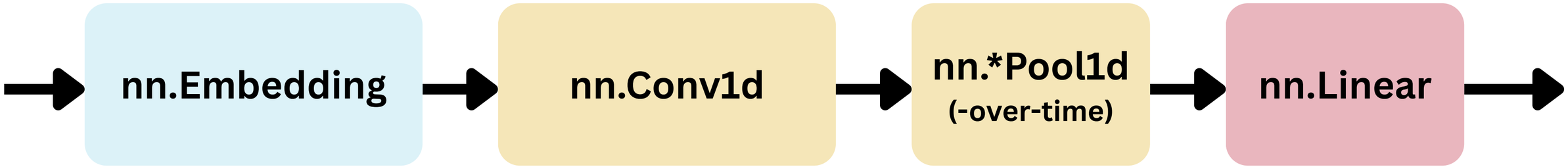
PyTorch: Train/Test Forward Pass

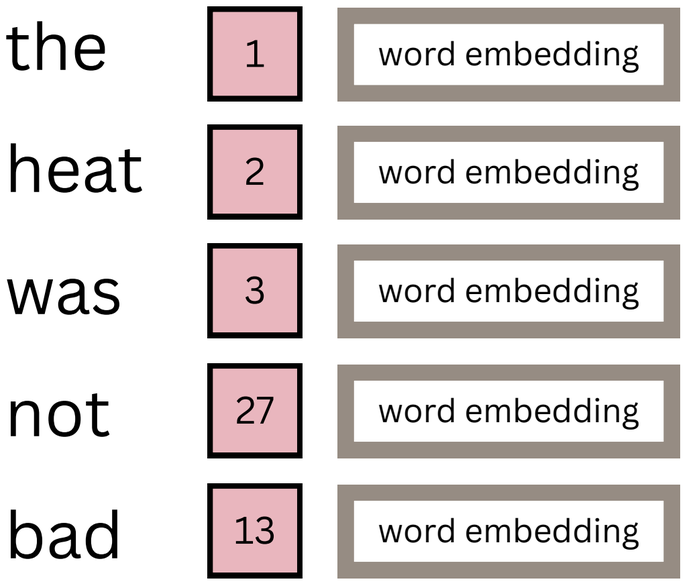
```
def train(...): # or def test
    pred = model(x_batch)
```



PyTorch: Train/Test Forward Pass

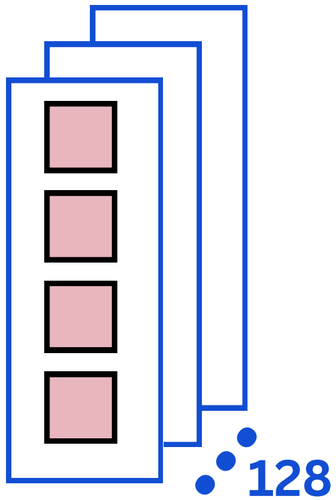
```
def train(...): # or def test
    pred = model(x_batch)
```





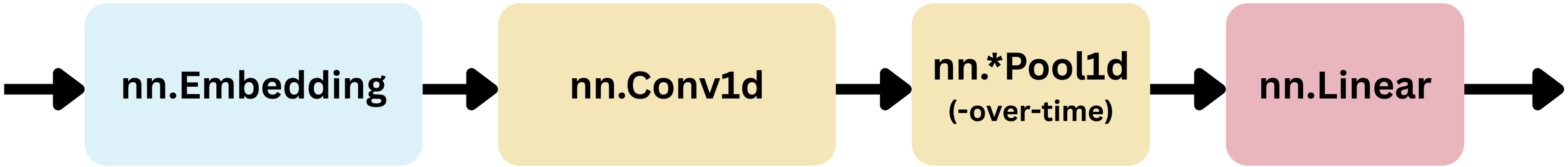
n_in=5
p=0
f=2
s=1
→ **n_out = 4**

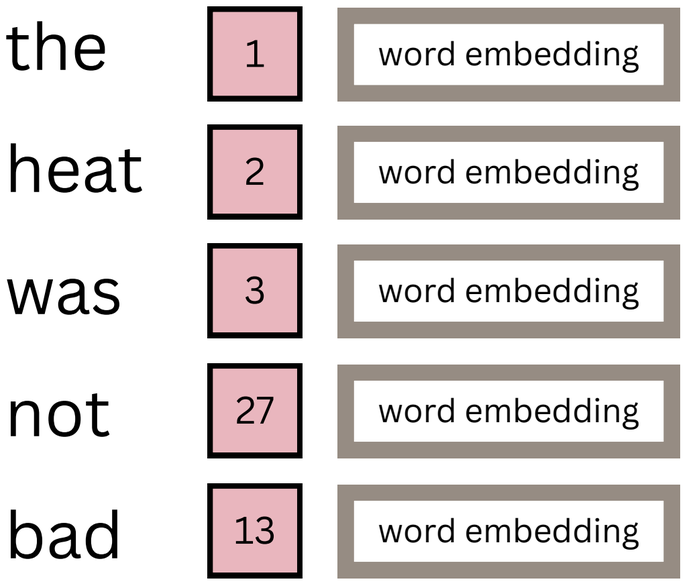
in_ch = 300
→ out_ch = 128



PyTorch: Train/Test Forward Pass

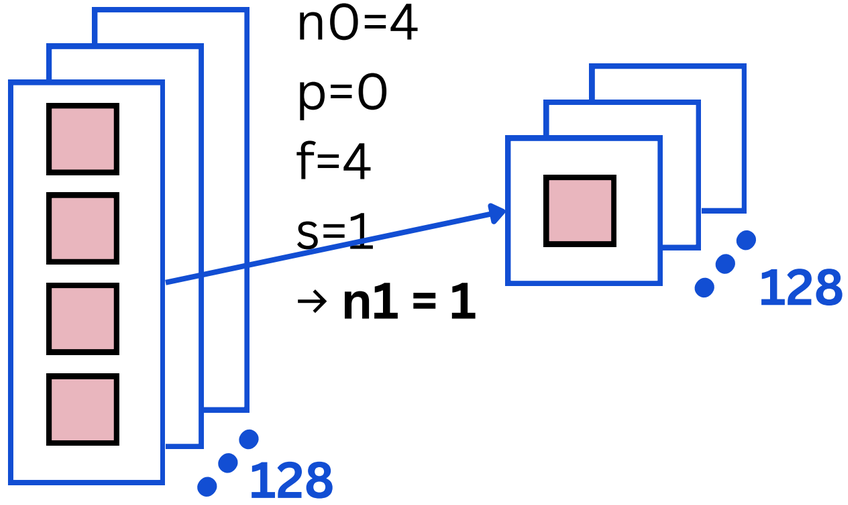
```
def train(...): # or def test
    pred = model(x_batch)
```





n_in=5
p=0
f=2
s=1
→ **n_out = 4**

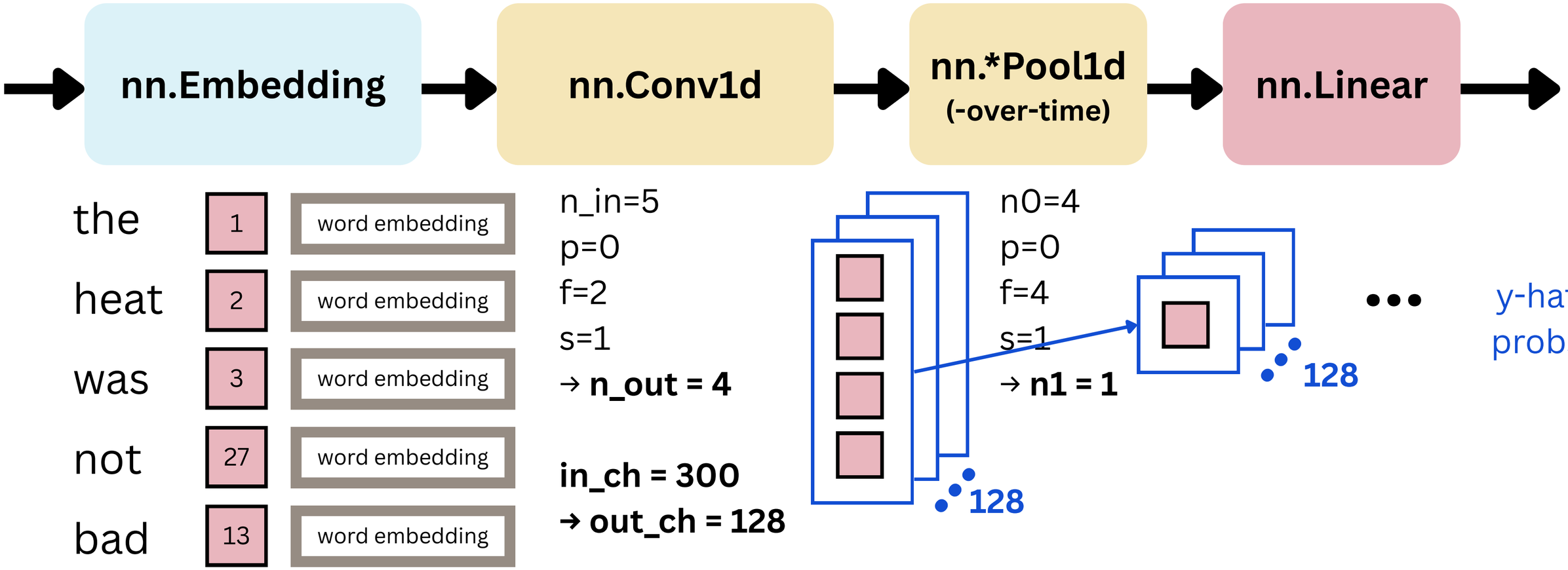
in_ch = 300
→ out_ch = 128



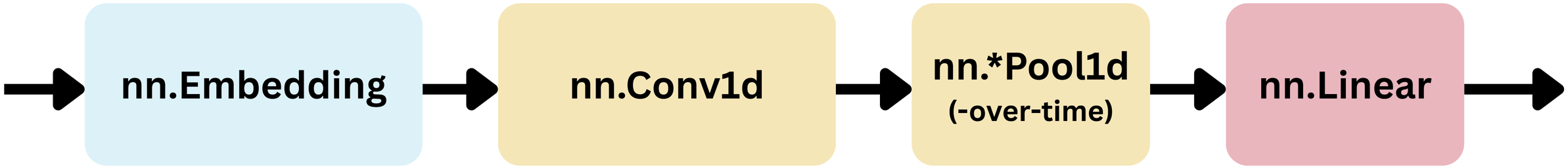
n0=4
p=0
f=4
s=1
→ **n1 = 1**

PyTorch: Train/Test Forward Pass

```
def train(...): # or def test
    pred = model(x_batch)
```



PyTorch: Training Back Propagation




```
def train(...):
    ...
    loss.backward() # Compute gradients
    optimizer.step() # Update parameters
    optimizer.zero_grad() # Reset gradients
```

y-hat
probs

Recurrent Neural Network (RNN)

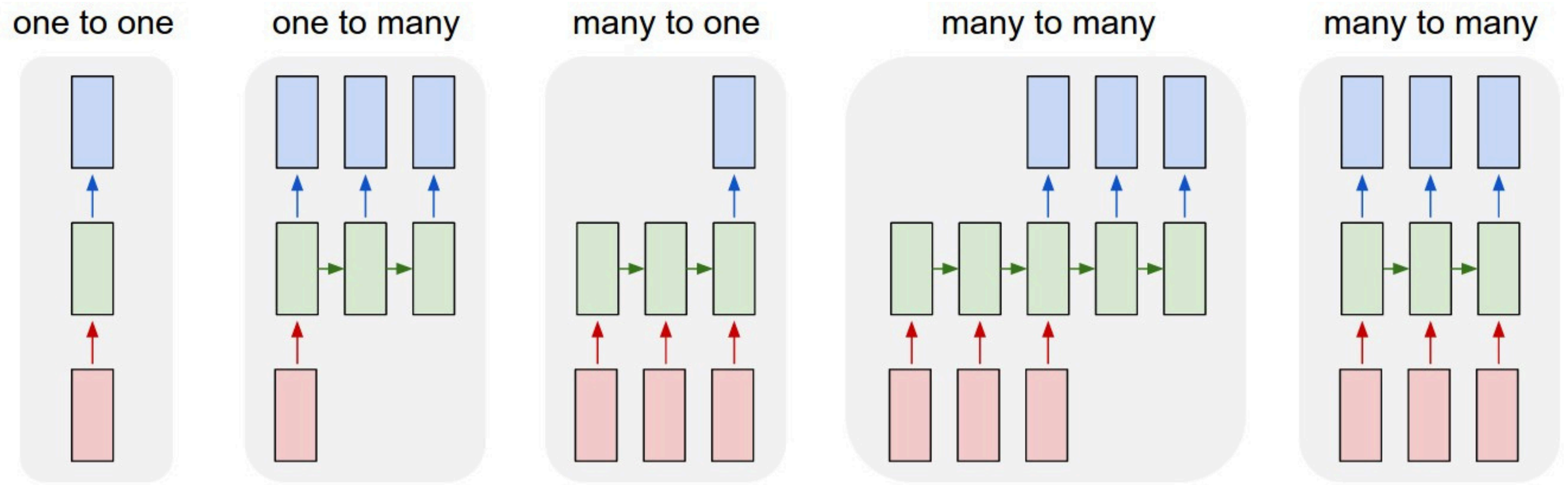
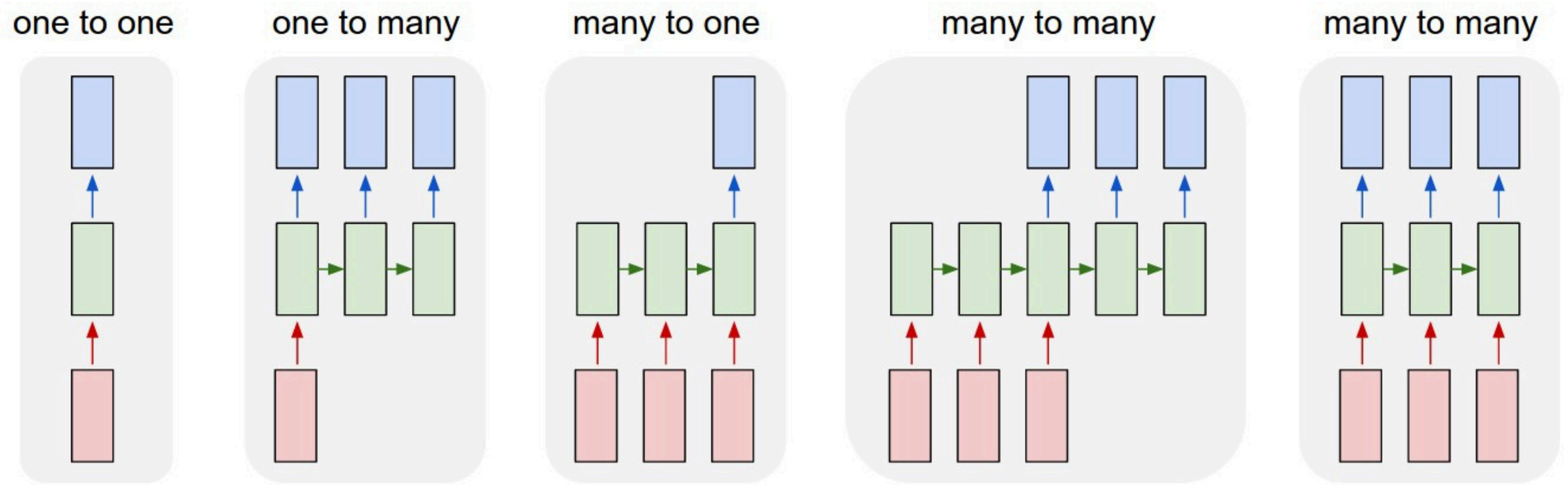


Image: Karpathy, 2015

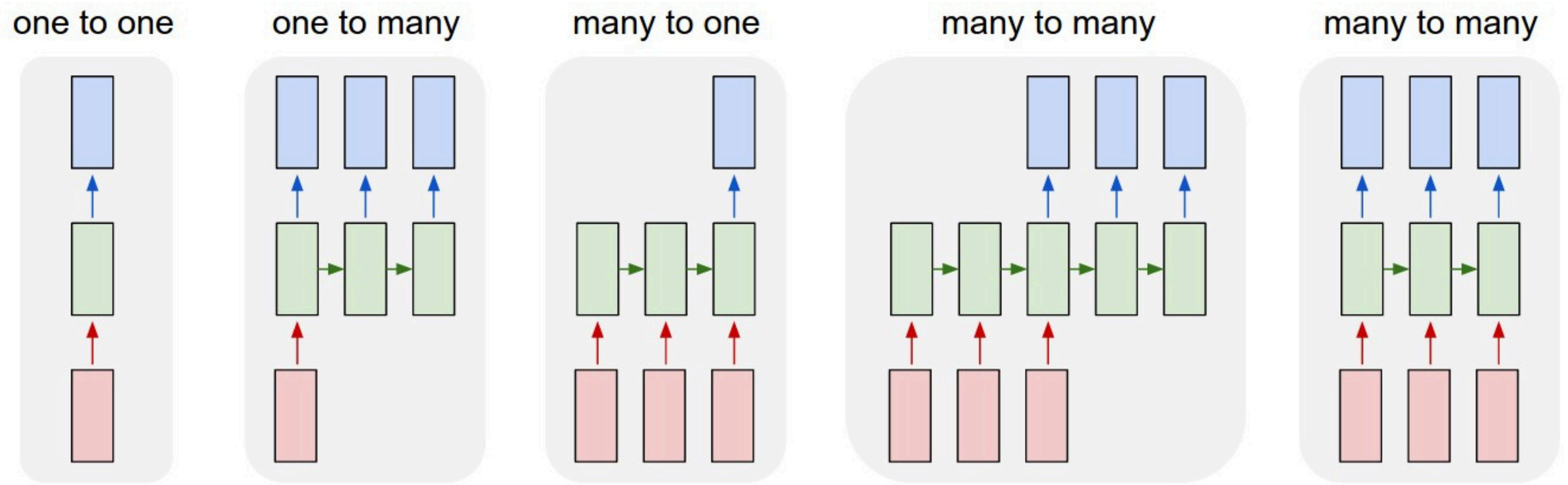
Recurrent Neural Network (RNN)



Classification

Image: Karpathy, 2015

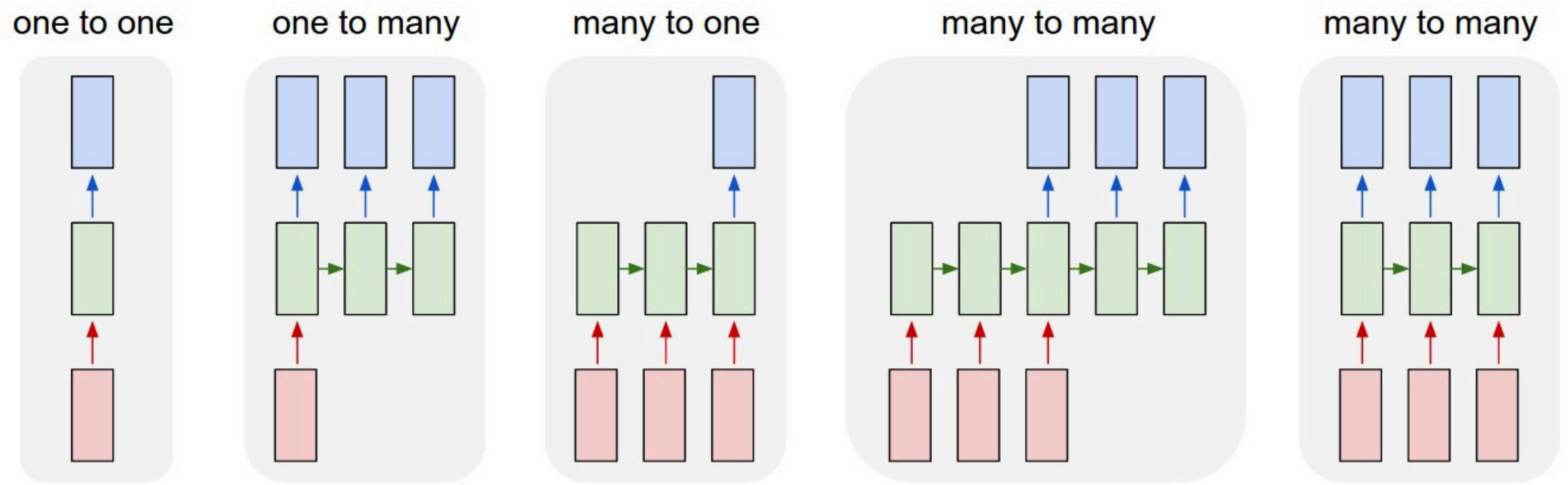
Recurrent Neural Network (RNN)



Classification **Translation**

Image: Karpathy, 2015

Recurrent Neural Network (RNN)



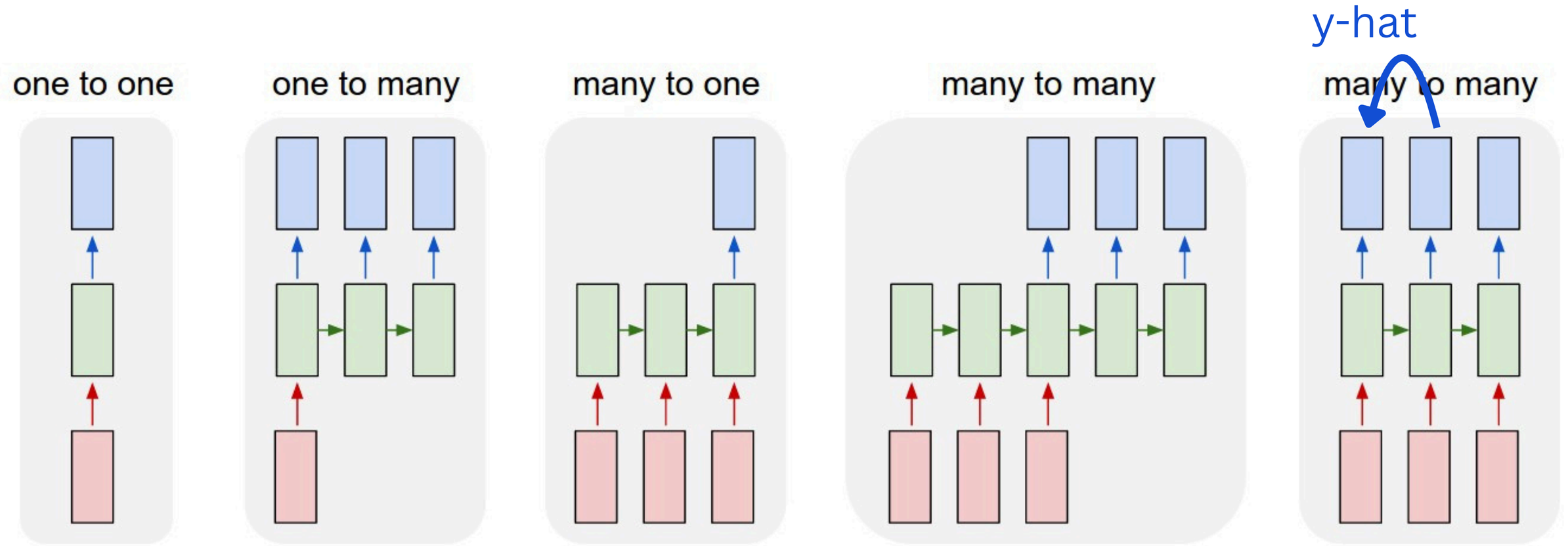
Classification

Translation

Next word prediction

Image: Karpathy, 2015

Recurrent Neural Network (RNN)



Classification

Translation

Next word prediction

Image: Karpathy, 2015

Recurrent Neural Network (RNN)

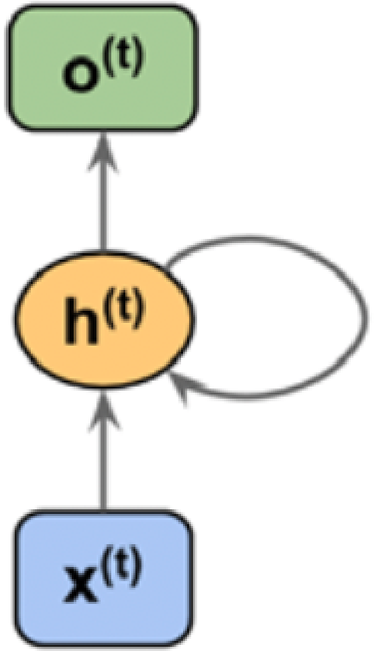


Image: Raschka, 2022

Recurrent Neural Network

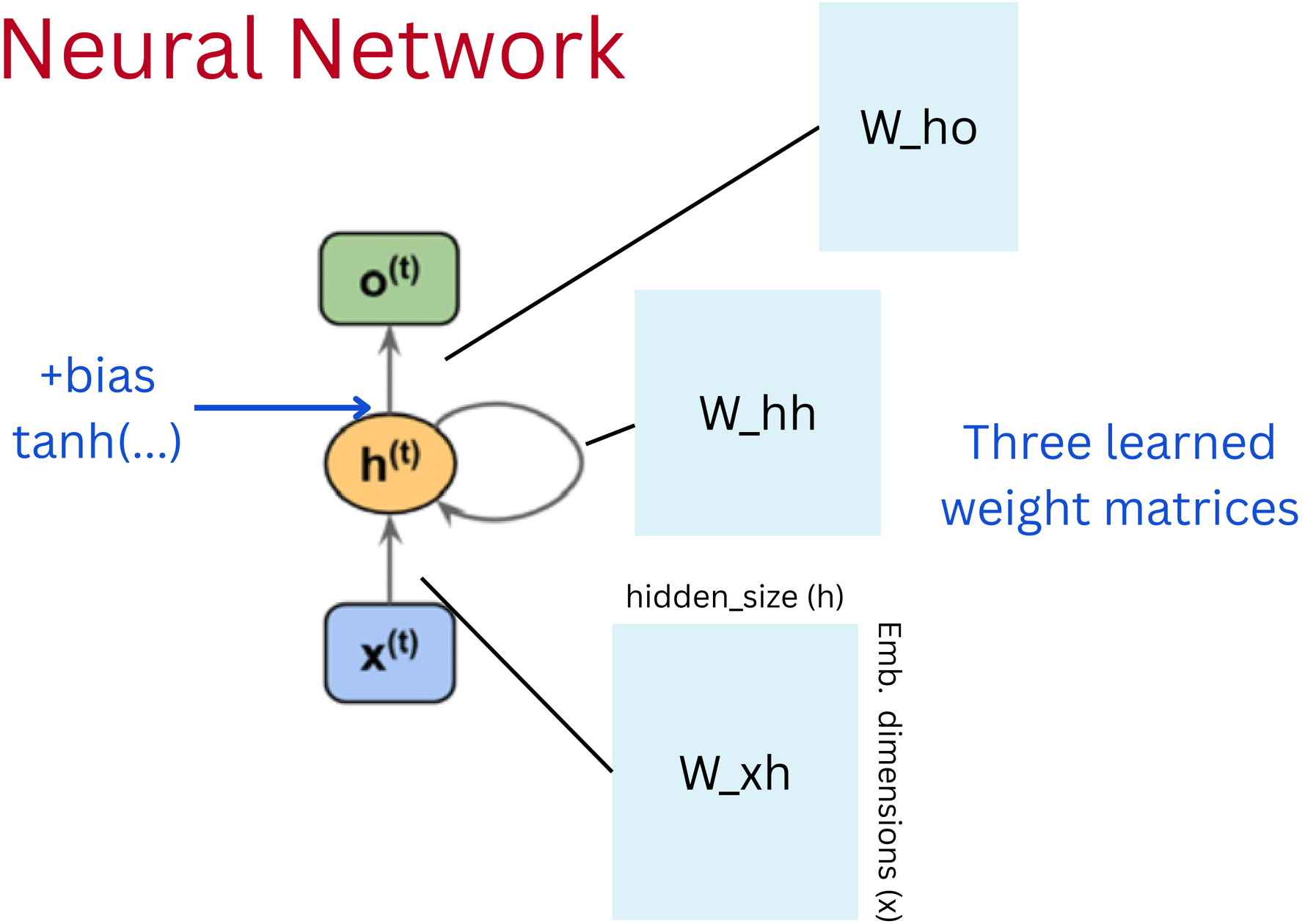


Image: Raschka, 2022

Recurrent Neural Network (RNN)

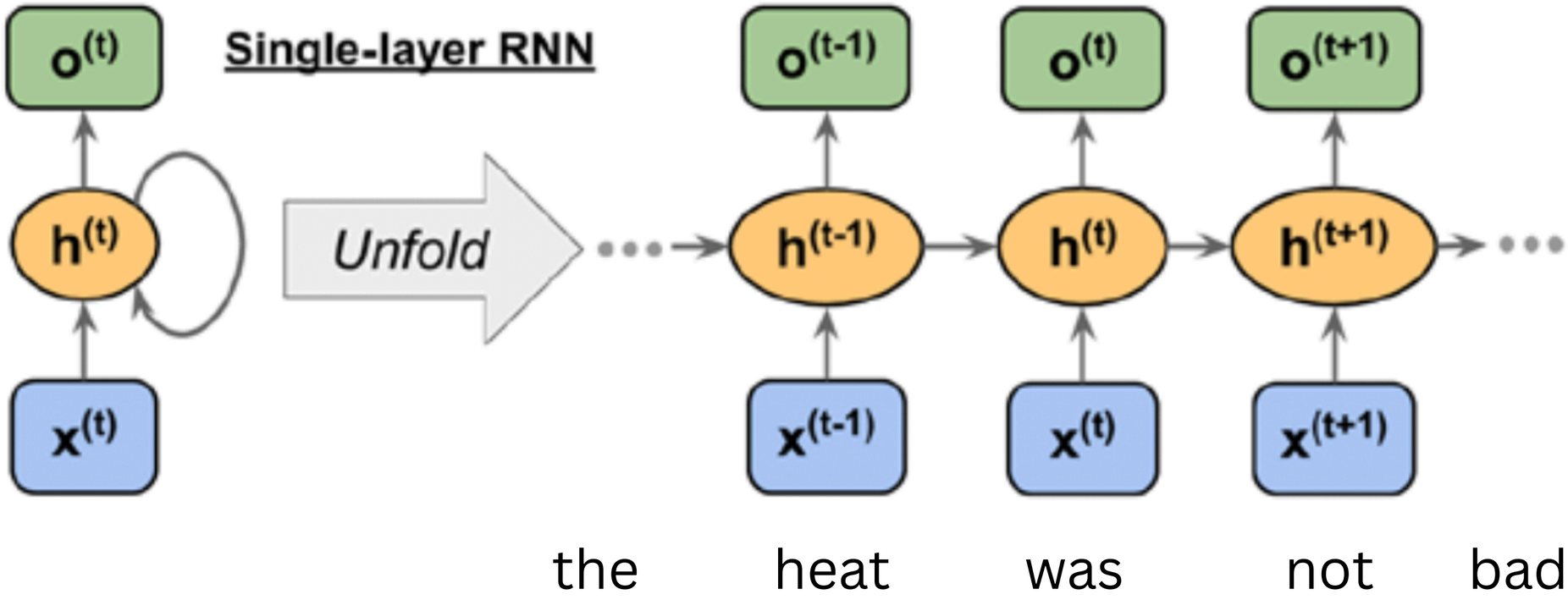
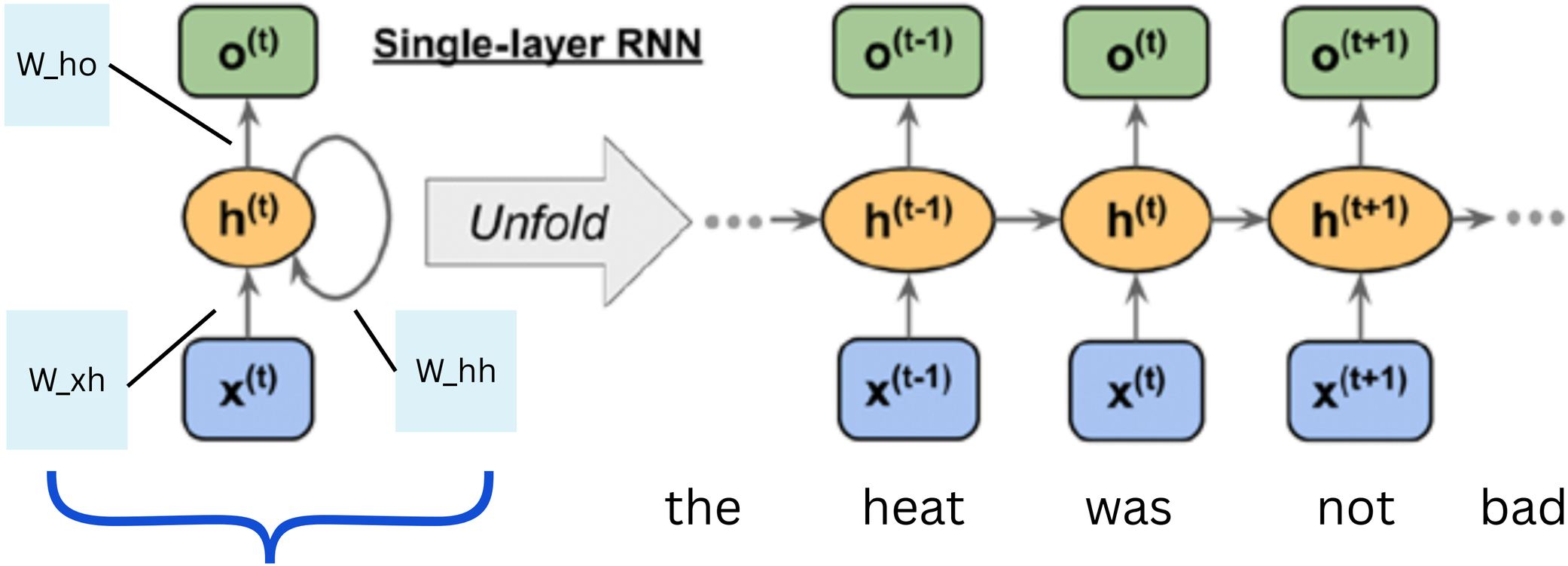


Image: Raschka, 2022

Recurrent Neural Network (RNN)



SAME set of weight matrices
applied at each time/text step

Image: Raschka, 2022

Recurrent Neural Network (RNN)

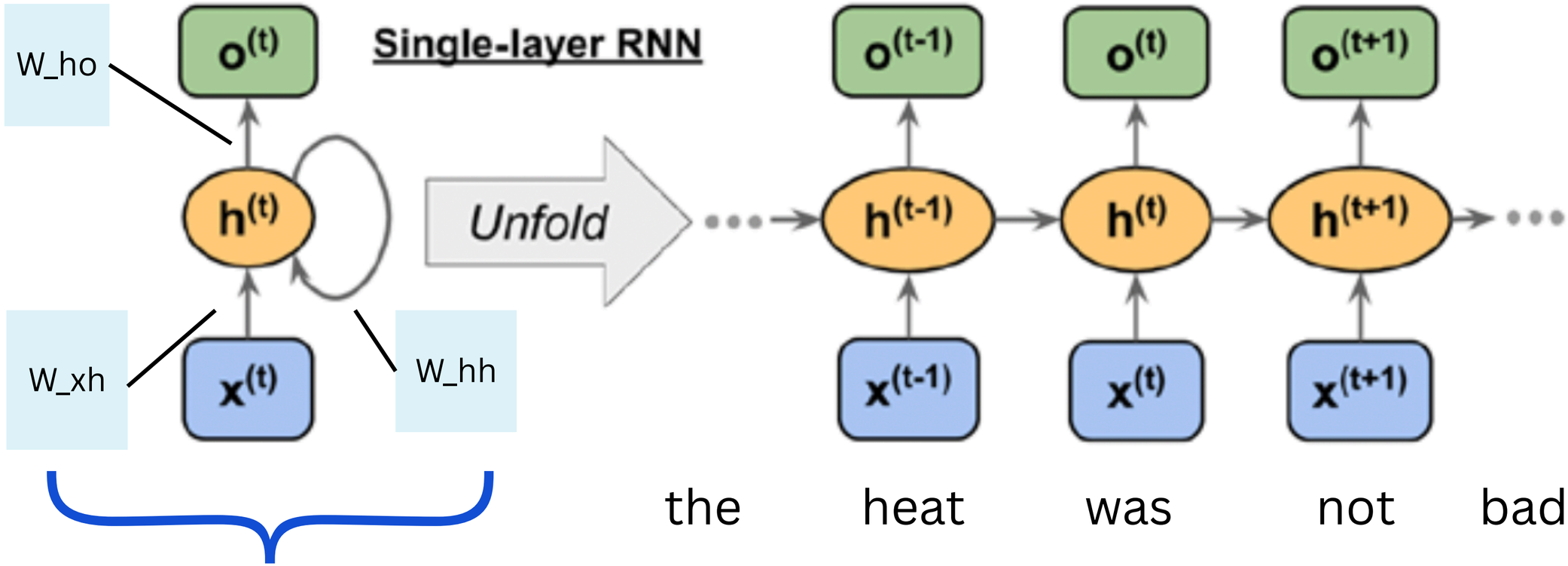
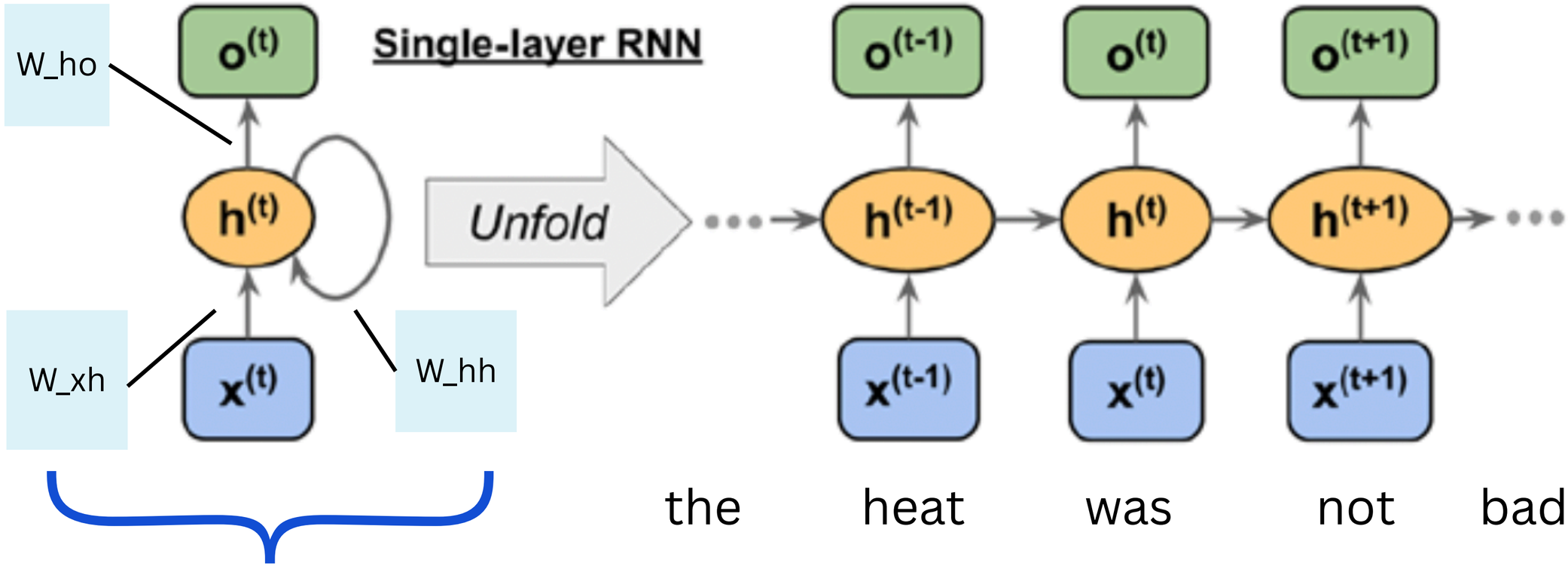


Image: Raschka, 2022

Recurrent Neural Network (RNN)



SAME set of weight matrices applied at each time/text step

Back propagation through time (BPTT)

Uses sum of all loss functions in sequence

Image: Raschka, 2022

Next Week: LSTM and Transformers!

